

Linear algebra in Python

- Most of the functions about matrix operations are included in numpy and scipy. You can find lots of tutorials online, e.g., the following links.
- <https://github.com/lijin-THU/notes-python/tree/master/03-numpy>
- <https://github.com/lijin-THU/notes-python/tree/master/04-scipy>
- Basic operations like matrix multiplication, LU decomposition, determinant, inverse, eigenvalue and eigenvector, solving linear equation, etc.

Example of codes

```
import numpy as np
import scipy

np.set_printoptions(precision=3,suppress=True)

#set up a new 2D array, matrix
A = np.array([[3, 8, 1, -4], [7, 3, -1, 2], [-1, 1, 4, -1], [2, -4, -1, 6] ])

b= np.array([3, 5, 7, 4])
```

```
#check the LU decomposition by using matrix product
print("check L*U, which should be A")
print(np.dot(A_L,A_U))
print()

#calculate the inverse of A
A_Inv=np.linalg.inv(A)
print("check A_inv*R, which should be I")
print(np.dot(A_Inv,A))
print()

#calculate the determinant of A
A_Det=np.linalg.det(A)
print("the determinant of A is", A_Det)
print()
```

```
#do the QR decomposition of A
A_Q, A_R = np.linalg.qr(A)

#check the QR decomposition by using matrix product
print("check Q*R, which should be A")
print(np.dot(A_Q,A_R))
print()
```

```
#calculate the eigen values and eigen vectors of A
A_x, A_V = np.linalg.eig(A)
print("the eigenvalues of A is", A_x)
print("the eigenvectors of A is", A_V)
#check the results of eigen values and eigen vectors
A_D=np.zeros([4,4],float)
for n in range(4):
    A_D[n,n]=np.real(A_x[n])
print()
print("check AV=VD, which should be zero")
print(np.dot(A,A_V)-np.dot(A_V,A_D))
print()
```

```
#solving the simultaneous linear equation Ax=b
x=np.linalg.solve(A,b)
#check the results
print("check Ax=b, which should be zero")
print(np.dot(A,x)-b)
print()
```

Be careful about the variable type

```
#Becareful all about the usage of array
#try to understand the following results
print("Becareful all about the usage of array")
print("A[0,0] = ", A[0,0])
print("A[0,0]/2 = ", A[0,0]/2, ", the result is right")
print("A[0,0]/2.0 = ", A[0,0]/2.0, ", the result is right")
print()
B=np.copy(A)
print("If we use B=copy(A) and B[0,0]=A[0,0]/2")
B[0,0]=A[0,0]/2
print("we get B[0,0] = ", B[0,0], ", the result is WRONG")
print()
print("If we use B=copy(A) and B[0,0]=A[0,0]/2.0")
B[0,0]=A[0,0]/2.0
print("we get B[0,0] = ", B[0,0], ", the result is WRONG")
print()
print("If we use B=copy(A) and B[0,0]=A[0,0]*1.0/2.0")
B[0,0]=A[0,0]*1.0/2.0
print("we get B[0,0] = ", B[0,0], ", the result is WRONG")
print()

B=np.copy(A)*1.0
print("If we use B=copy(A)*1.0 and B[0,0]=A[0,0]/2")
B[0,0]=A[0,0]/2
print("we get B[0,0] = ", B[0,0], ", the result is RIGHT")
```

Eigenvalues and eigenvectors

- One common problem is the calculation of the eigenvalues and/or eigenvectors of a matrix.
- For a matrix A , an eigenvector $\mathbf{v}$ is a vector satisfying $A\mathbf{v} = \lambda\mathbf{v}$, where λ is the corresponding eigenvalues. For a $N \times N$ matrix there are N eigenvectors with eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_N$. The eigenvectors have the property that they are orthogonal to one another $\mathbf{v}_i \cdot \mathbf{v}_j = 0$ if $i \neq j$, and we will assume they are normalized to have unit length $\mathbf{v}_i \cdot \mathbf{v}_i = 1$.
- If we wish, we can consider the eigenvectors to be the columns of a single $N \times N$ matrix V and combine all the equations $A\mathbf{v}_i = \lambda_i\mathbf{v}_i$ into a single matrix $AV = VD$, where D is the diagonal matrix with the eigenvalues λ_i as its diagonal entries. Notice that the matrix V is **orthogonal**, meaning that its transpose V^T is equal to its inverse V^{-1} , so that $V^T V = VV^T = I$.

How to get the eigenvalues and eigenvectors?

- Based on the definition, for a matrix A , an eigenvalue λ and an eigenvector $\mathbf{v}$ obey the equation $A\mathbf{v} = \lambda\mathbf{v}$, and it can be written as $(A - \lambda I)\mathbf{v} = \mathbf{0}$, which is a set of simultaneous linear equations with the form $B\mathbf{v} = \mathbf{0}$ (we take $B = A - \lambda I$). It seems that we can use Gaussian elimination to solve it. **Can we? why?**
- The answer is NO, because there are unknown variables in the matrix B but not only in the vector $\mathbf{v}$. And there is always a trivial solution $\mathbf{v} = (0, 0, \dots, 0)$. So if there is only solution of the equation, then $\mathbf{v} = (0, 0, \dots, 0)$. Thus, the actual question should be what values of λ can make the equation $(A - \lambda I)\mathbf{v} = \mathbf{0}$ have non-zero solution $\mathbf{v}$.
- The answer is that if and only if the determinant of $(A - \lambda I)$ is zero, then there will non-zero solution $\mathbf{v}$ for $(A - \lambda I)\mathbf{v} = \mathbf{0}$.
- Finally, we can get the eigenvalues by solving the following nonlinear equation $\det(A - \lambda I) = 0$

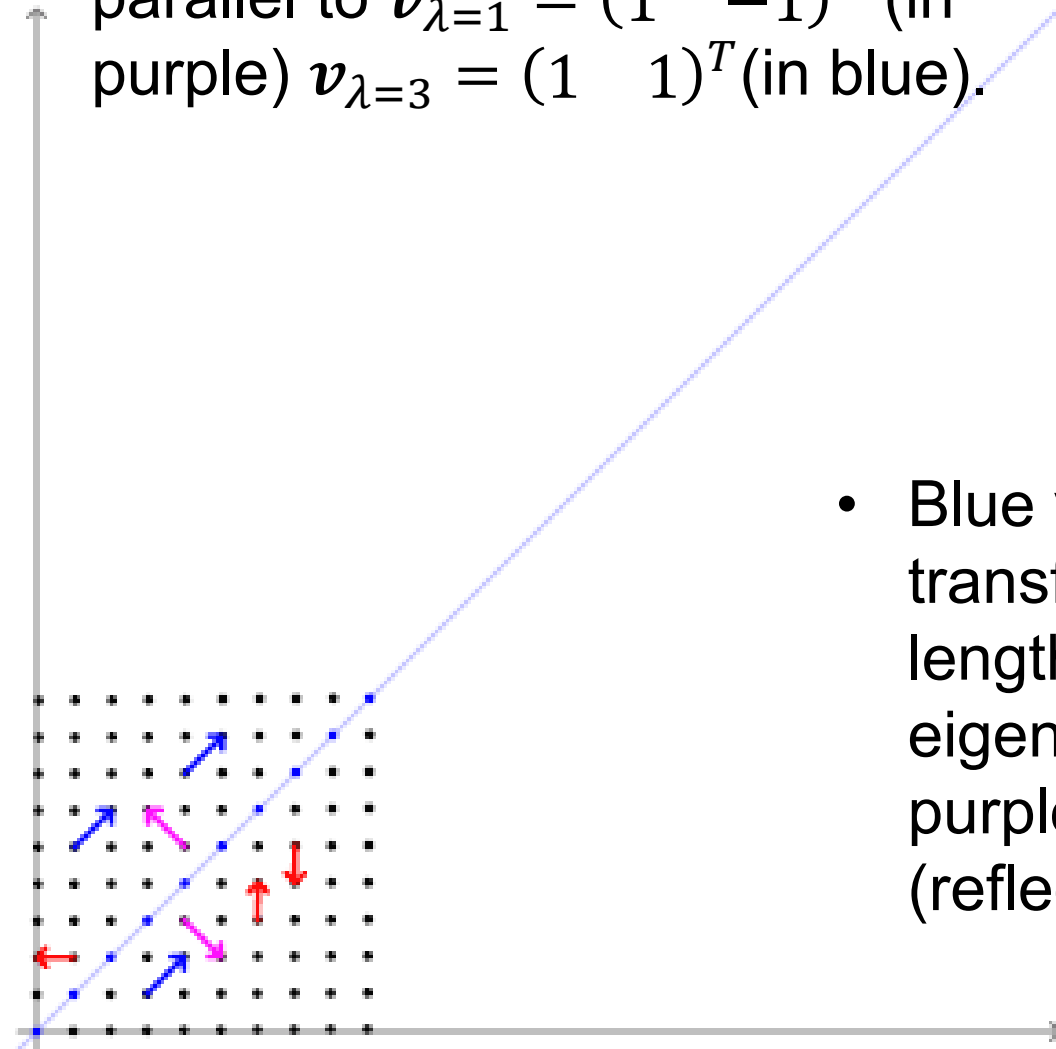
An example

- For a diagonal matrix, the answer is very simple. The eigenvalues are just the diagonal elements.
- Consider a matrix $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$
- Taking the determinant to find the characteristic polynomial of A
 $|A - \lambda I| = \left| \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} - \lambda \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right| = \begin{vmatrix} 2 - \lambda & 1 \\ 1 & 2 - \lambda \end{vmatrix} = 3 - 4\lambda + \lambda^2 = 0$
- Then we get the eigenvalues are $\lambda = 1$ and $\lambda = 3$
- Next, we will find the eigenvectors. For $\lambda = 1$, $(A - \lambda I)\mathbf{v} = \mathbf{0}$ becomes $\begin{pmatrix} 2 - 1 & 1 \\ 1 & 2 - 1 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$, and we can easily get
$$\mathbf{v}_{\lambda=1} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$
- Similarly, we can get the eigenvector corresponding to $\lambda = 3$
$$\mathbf{v}_{\lambda=3} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

The meaning of eigenvalues and eigenvectors

- The transformation matrix A preserves the direction of vectors parallel to $v_{\lambda=1} = (1 \ -1)^T$ (in purple) $v_{\lambda=3} = (1 \ 1)^T$ (in blue).
- Vectors in red are not parallel to either eigenvector, so their directions are changed by the transformation.

- Blue vectors after the transformation are three times the length of the original (their eigenvalue is 3), while lengths of purple vectors are unchanged (reflecting an eigenvalue of 1).



Solve eigenequation by QR decomposition

- The calculation of eigenvalues and eigenvectors based on the determinant involves solving nonlinear equations, and it is hard to implement in codes.
- There are many other algorithms to do the calculations. One of the most used method is QR decomposition. QR decomposition is a variant on the same ideas as LU decomposition, but now the matrix is written as the product QR of an orthogonal matrix Q and an upper-triangular matrix R . Any square matrix can be written in this form.
- Suppose we have a real, square matrix A and let us break it down into its decomposition as $A = Q_1 R_1$, then we get

$$Q_1^T A = Q_1^T Q_1 R_1 = R_1$$

, where we have used the fact that Q_1 is orthogonal ($Q_1^T Q_1 = I$)

- Now we define a new matrix $A_1 = R_1 Q_1$, which is just the product of the same two matrices, but in the reverse order. Due to $R_1 = Q_1^T A$, we have $A_1 = Q_1^T A Q_1$

Solve eigenequation by QR decomposition

- Using an orthogonal matrix Q_1 , we get a new matrix $A_1 = Q_1^T A Q_1$
- Now we do the QR decomposition for matrix $A_1 = Q_2 R_2$, and get another new matrix $A_2 = R_2 Q_2 = Q_2^T A_1 Q_2 = Q_2^T Q_1^T A Q_1 Q_2$
- We can do it again and again, repeatedly forming the QR decomposition of the current matrix, then multiplying R and Q in the opposite order to get a new matrix. If we do a total of steps, we generate the sequence of matrices

$$A_1 = Q_1^T A Q_1$$

$$A_2 = Q_2^T Q_1^T A Q_1 Q_2$$

$$A_3 = Q_3^T Q_2^T Q_1^T A Q_1 Q_2 Q_3$$

$$\vdots$$

$$A_k = (Q_k^T \cdots Q_1^T) A (Q_1 \cdots Q_k)$$

- It can be proven that if you continue this process long enough, the matrix A_k will eventually become diagonal. The off-diagonal entries of the matrix get smaller and smaller the more iterations of the process you do.

Eigenvalue decomposition (EVD) of a matrix

- Suppose we continue until all the off-diagonal elements become so small that, to whatever accuracy we choose, the matrix A_k approximates a diagonal matrix D . Let us define

$$V = Q_1 Q_2 Q_3 \cdots Q_k = \prod_{i=1}^k Q_i$$

, which is an orthogonal matrix since a product of orthogonal matrices is always another orthogonal matrix. Then we have

$$D = A_k = (Q_k^T \cdots Q_1^T) A (Q_1 \cdots Q_k) = V^T A V$$

- And multiplying on the left by V , we get

$$A V = V D$$

- This is precisely the definition of the eigenvalues and eigenvectors of A in the matrix form. Thus, the columns of V are the eigenvectors of A and the diagonal elements of D are the corresponding eigenvalues (in the same order as the eigenvectors).

Detailed procedures of the complete algorithm

- For a symmetric $N \times N$ matrix A , the algorithm is as follows:
 1. Create an $N \times N$ matrix V to hold the eigenvalues and initially set it equal to the identity matrix I . Also choose a target accuracy ϵ for the off-diagonal elements of the eigenvalue matrix.
 2. Calculate the QR decomposition $A = QR$.
 3. Update A to the new value $A = RQ$.
 4. Multiply V on the right by Q .
 5. Check the off-diagonal elements of A . If they are all less than ϵ , we are done. Otherwise go back to step 2.
- When the algorithm ends, the diagonal elements of A contain the eigenvalues and the columns of V contain the eigenvectors.
- The calculations for a general matrix are very subtle. More details in <https://www.andreinc.net/2021/01/25/computing-eigenvalues-and-eigenvectors-using-qr-decomposition>.

Gram-Schmidt algorithm for QR decomposition

- Consider a matrix $A = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{pmatrix}$. We can think of the matrix as a set of N column vectors $\vec{a}_1, \vec{a}_2, \dots, \vec{a}_n$, thus $A = (\vec{a}_1, \vec{a}_2, \dots, \vec{a}_n)$ with $\vec{a}_i = (a_{1i}, a_{2i}, \dots, a_{ni})^T$
- We can define two new sets of vectors $\vec{u}_1, \vec{u}_2, \dots, \vec{u}_n$ and $\vec{q}_1, \vec{q}_2, \dots, \vec{q}_n$ as follows:

$$\vec{u}_1 = \vec{a}_1,$$

$$\vec{q}_1 = \frac{\vec{u}_1}{|\vec{u}_1|}$$

$$\vec{u}_2 = \vec{a}_2 - (\vec{q}_1 \cdot \vec{a}_2)\vec{q}_1,$$

$$\vec{q}_2 = \frac{\vec{u}_2}{|\vec{u}_2|}$$

$$\vec{u}_3 = \vec{a}_3 - (\vec{q}_1 \cdot \vec{a}_3)\vec{q}_1 - (\vec{q}_2 \cdot \vec{a}_3)\vec{q}_2,$$

$$\vec{q}_3 = \frac{\vec{u}_3}{|\vec{u}_3|}$$

- The general formulas for calculating $\vec{u}_i$ and $\vec{q}_i$ are

$$\vec{u}_i = \vec{a}_i - \sum_{j=1}^{i-1} (\vec{q}_j \cdot \vec{a}_i)\vec{q}_j, \quad \vec{q}_i = \frac{\vec{u}_i}{|\vec{u}_i|}$$

Gram-Schmidt algorithm for QR decomposition

- We can demonstrate that, all the vectors $\vec{q}_i$ are orthonormal, i.e., that they satisfy $\vec{q}_i \cdot \vec{q}_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$
- Now, rearranging the definitions of the vectors, we have

$$\vec{a}_1 = |\vec{u}_1| \vec{q}_1,$$

$$\vec{a}_2 = |\vec{u}_2| \vec{q}_2 + (\vec{q}_1 \cdot \vec{a}_2) \vec{q}_1,$$

$$\vec{a}_3 = |\vec{u}_3| \vec{q}_3 + (\vec{q}_1 \cdot \vec{a}_3) \vec{q}_1 + (\vec{q}_2 \cdot \vec{a}_3) \vec{q}_2,$$

- We can group the vectors together as the columns of a matrix and write all of these equations as a single matrix equation

$$\begin{aligned} A = (\vec{a}_1, \vec{a}_2, \vec{a}_3, \dots) &= (\vec{q}_1, \vec{q}_2, \vec{q}_3, \dots) \begin{pmatrix} |\vec{u}_1| & (\vec{q}_1 \cdot \vec{a}_2) & (\vec{q}_1 \cdot \vec{a}_3) & \dots \\ 0 & |\vec{u}_2| & (\vec{q}_2 \cdot \vec{a}_3) & \dots \\ 0 & 0 & |\vec{u}_3| & \dots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix} \\ &= QR \end{aligned}$$

Gram-Schmidt algorithm for QR decomposition

- For $A = (\vec{a}_1, \vec{a}_2, \dots, \vec{a}_n)$ with $\vec{a}_i = (a_{1i}, a_{2i}, \dots, a_{ni})^T$
- We define $\vec{u}_i$ and $\vec{q}_i$ as

$$\vec{u}_i = \vec{a}_i - \sum_{j=1}^{i-1} (\vec{q}_j \cdot \vec{a}_i) \vec{q}_j, \quad \vec{q}_i = \frac{\vec{u}_i}{|\vec{u}_i|}$$

- We can demonstrate

$$A = (\vec{a}_1, \vec{a}_2, \vec{a}_3, \dots) = (\vec{q}_1, \vec{q}_2, \vec{q}_3 \dots) \begin{pmatrix} |\vec{u}_1|(\vec{q}_1 \cdot \vec{a}_2)(\vec{q}_1 \cdot \vec{a}_3) \cdots \\ 0 & |\vec{u}_2| & (\vec{q}_2 \cdot \vec{a}_3) \cdots \\ 0 & 0 & |\vec{u}_3| & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix}$$

- Thus $Q = (\vec{q}_1, \vec{q}_2, \vec{q}_3 \dots)$ and $R = \begin{pmatrix} |\vec{u}_1|(\vec{q}_1 \cdot \vec{a}_2)(\vec{q}_1 \cdot \vec{a}_3) \cdots \\ 0 & |\vec{u}_2| & (\vec{q}_2 \cdot \vec{a}_3) \cdots \\ 0 & 0 & |\vec{u}_3| & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix}$,
and $A = QR$.

An example

$$A = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 1 & 2 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ -1 & 0 & 1 & 0 \end{pmatrix}$$

- Based on the definition of $\vec{u}_i$ and $\vec{q}_i$

$$\vec{u}_i = \vec{a}_i - \sum_{j=1}^{i-1} (\vec{q}_j \cdot \vec{a}_i) \vec{q}_j, \quad \vec{q}_i = \frac{\vec{u}_i}{|\vec{u}_i|}$$

- First, we get all the $\vec{u}_i$ and $\vec{q}_i$ by the definition

$$\begin{cases} \vec{u}_1 = \vec{a}_1 = (1, 1, 1, -1)^T \\ \vec{q}_1 = \vec{u}_1 / |\vec{u}_1| = (1, 1, 1, -1)^T / 2 \end{cases}$$

$$\begin{cases} \vec{u}_2 = \vec{a}_2 - (\vec{q}_1 \cdot \vec{a}_2) \vec{q}_1 = (2, 2, 0, 0)^T - (1, 1, 1, -1)^T \\ \quad = (1, 1, -1, 1)^T \\ \vec{q}_2 = \vec{u}_2 / |\vec{u}_2| = (1, 1, -1, 1)^T / 2 \end{cases}$$

$$\begin{cases} \vec{u}_3 = \vec{a}_3 - (\vec{q}_2 \cdot \vec{a}_3) \vec{q}_2 - (\vec{q}_1 \cdot \vec{a}_3) \vec{q}_1 \\ \quad = (3, 1, 1, 1)^T - (1, 1, -1, 1)^T - (1, 1, 1, -1)^T \\ \quad = (1, -1, 1, 1)^T \\ \vec{q}_3 = \vec{u}_3 / |\vec{u}_3| = (1, -1, 1, 1)^T / 2 \end{cases}$$

$$\begin{cases} \vec{u}_4 = \vec{a}_4 - (\vec{q}_3 \cdot \vec{a}_4) \vec{q}_3 - (\vec{q}_2 \cdot \vec{a}_4) \vec{q}_2 - (\vec{q}_1 \cdot \vec{a}_4) \vec{q}_1 \\ \quad = (4, 0, 0, 0)^T - (1, -1, 1, 1)^T - (1, 1, -1, 1)^T \\ \quad \quad - (1, 1, 1, -1)^T = (1, -1, -1, -1)^T \\ \vec{q}_4 = \vec{u}_4 / |\vec{u}_4| = (1, -1, -1, -1)^T / 2 \end{cases}$$

- Finally, we can get Q and R

$$Q = (\vec{q}_1, \vec{q}_2, \vec{q}_3 \dots)$$

$$= \begin{pmatrix} 0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & -0.5 & -0.5 \\ 0.5 & -0.5 & 0.5 & -0.5 \\ -0.5 & 0.5 & 0.5 & -0.5 \end{pmatrix}$$

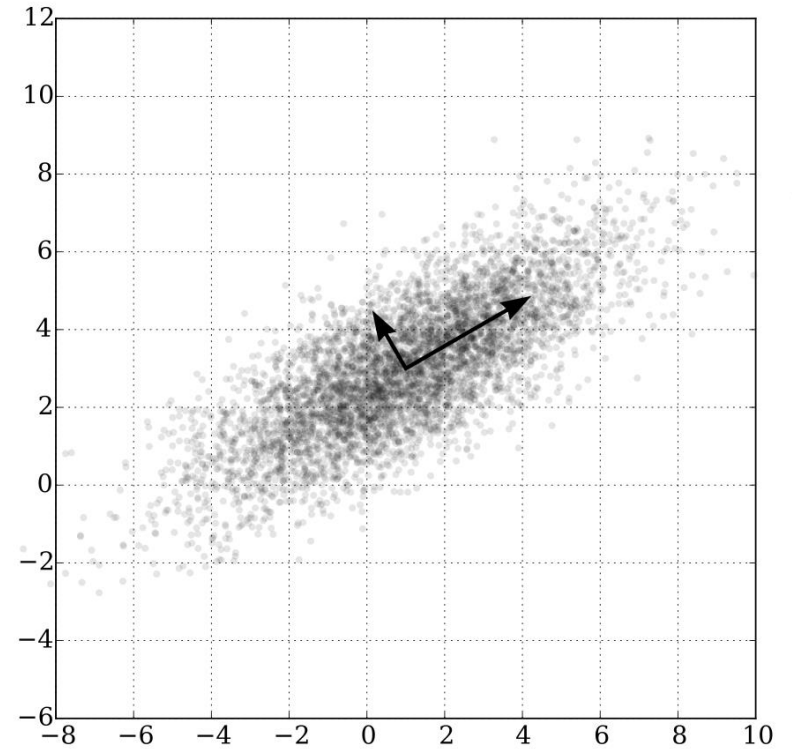
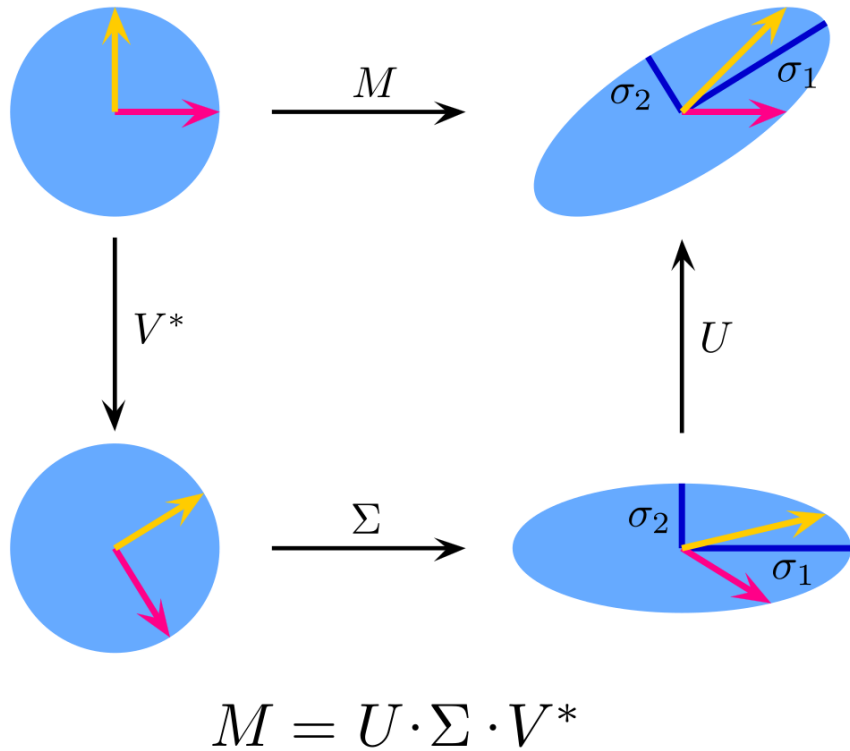
$$R = \begin{pmatrix} |\vec{u}_1| (\vec{q}_1 \cdot \vec{a}_2) (\vec{q}_1 \cdot \vec{a}_3) (\vec{q}_1 \cdot \vec{a}_4) \\ 0 & |\vec{u}_2| & (\vec{q}_2 \cdot \vec{a}_3) (\vec{q}_2 \cdot \vec{a}_4) \\ 0 & 0 & |\vec{u}_3| & (\vec{q}_3 \cdot \vec{a}_4) \\ 0 & 0 & 0 & |\vec{u}_4| \end{pmatrix}$$

$$= \begin{pmatrix} 2 & 2 & 2 & 2 \\ 0 & 2 & 2 & 2 \\ 0 & 0 & 2 & 2 \\ 0 & 0 & 0 & 2 \end{pmatrix}$$

SVD and PCA

- **SVD** and **PCA** are two concepts closely related to EVD, which are very useful in many areas, e.g. numerical stabilization, machine learning, signal processing, etc.
- **SVD** is the singular value decomposition, which can be regarded as an extension of EVD for non-square matrix of size $m \times n$. $\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$, where the diagonal entries of $\mathbf{D}$ are the singular values of $\mathbf{\Sigma}$, and the first $p = \min(m, n)$ columns of $\mathbf{U}$ and $\mathbf{V}$ are left- and right-singular vectors for the corresponding singular values.
- **PCA** refers to the principal component analysis, which is used to convert a set of possibly correlated variables into a set of linearly uncorrelated variables (principal components).

SVD and PCA

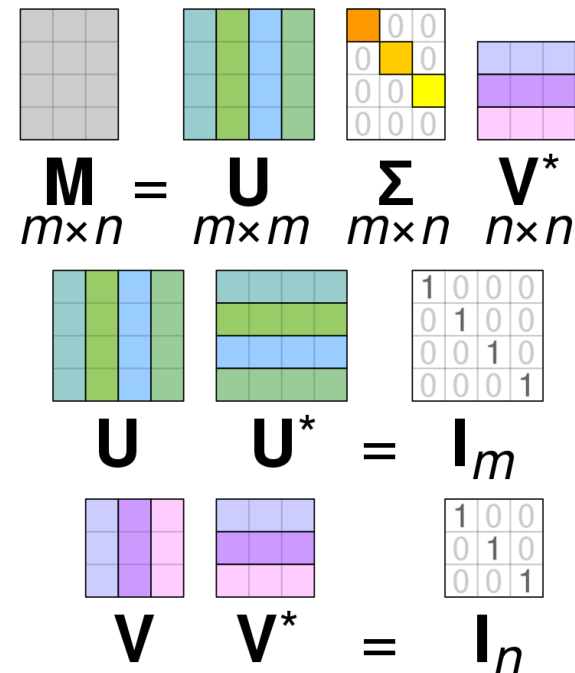
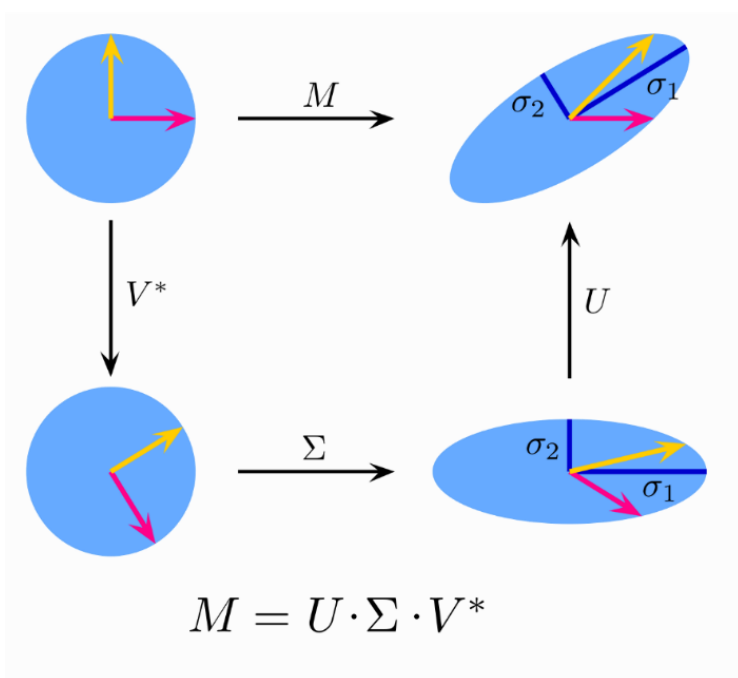


Singular Value Decomposition (SVD)

- In linear algebra, the singular value decomposition (SVD) is a factorization of a real or complex matrix that generalizes the eigendecomposition of a square normal matrix to any $m \times n$ matrix via an extension of the polar decomposition.
- Specifically, the singular value decomposition of an $m \times n$ real or complex matrix $\mathbf{M}$ is a factorization of the form $\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$, where $\mathbf{U}$ is an $m \times m$ real or complex **unitary matrix**, $\mathbf{\Sigma}$ is an $m \times n$ rectangular **diagonal matrix** with ***non-negative real numbers*** on the diagonal, and $\mathbf{V}$ is an $n \times n$ real or complex **unitary matrix**. If $\mathbf{M}$ is real, $\mathbf{U}$ and $\mathbf{V}^* = \mathbf{V}^T$ are real orthogonal matrices.
- The diagonal entries $\sigma_i = \Sigma_{ii}$ of $\mathbf{\Sigma}$ are known as the ***singular values*** of $\mathbf{M}$. The number of non-zero singular values is equal to the rank of $\mathbf{M}$. The columns of $\mathbf{U}$ and the columns of $\mathbf{V}$ are called the ***left-singular vectors*** and ***right-singular vectors*** of $\mathbf{M}$, respectively.
- The SVD is not unique. It is always possible to choose the decomposition so that the singular values Σ_{ii} are in descending order. In this case, $\mathbf{\Sigma}$ (but not always $\mathbf{U}$ and $\mathbf{V}$) is uniquely determined by $\mathbf{M}$.

Singular value decomposition

- SVD sometimes also refers to the compact SVD, a similar decomposition $\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$ in which $\mathbf{\Sigma}$ is square diagonal of size $r \times r$, where $r \leq \min\{m, n\}$ is the rank of $\mathbf{M}$, and has only the non-zero singular values. In this variant, $\mathbf{U}$ is an $m \times r$ semi-unitary matrix and $\mathbf{V}$ is an $n \times r$ semi-unitary matrix, such that $\mathbf{U}^* \mathbf{U} = \mathbf{V}^* \mathbf{V} = \mathbf{I}_{r \times r}$.
- The SVD is also extremely useful in all areas of science, engineering, and statistics, such as signal processing, least squares fitting of data, and process control.



Data compression

$$A_{M \times N} \approx U_{M \times r} \Sigma_{r \times r} V_{N \times N}^T$$

$$A_{M \times N} = U_{M \times M} * \Sigma_{M \times N} * V_{N \times N}^T$$

- The number of data you need to save is now $r * (M + N + 1)$, which is much smaller than the original $M * N$
- It is very useful in the image processing.

The example codes

```
from scipy import misc
import matplotlib.pyplot as plt
from numpy import linalg
import numpy as np
```

```
plt.figure()
img = misc.face()
img_array=img/img.max()
```

*#If our array has more than two dimensions, then the SVD can be applied to
#all axes at once. However, the linear algebra functions in NumPy expect to
#see an array of the form (N, :, :), where the first axis represents
#the number of matrices*

```
img_array_transposed = np.transpose(img_array, (2, 0, 1))
```

```
Ua, sa, Vta = linalg.svd(img_array_transposed)
```

```
Sigmaa = np.zeros((3, 768, 1024))
```

```
for j in range(3):
    np.fill_diagonal(Sigmaa[j,:,:], sa[j,:])
```

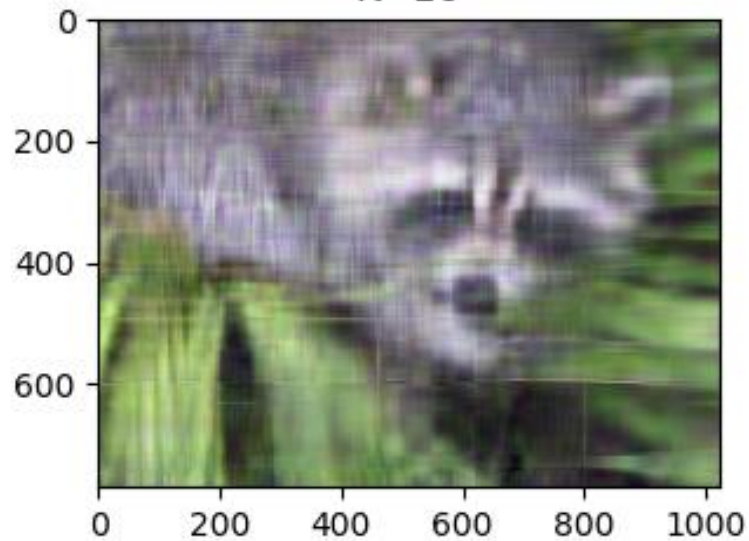
```
for k in (10,20,30,40,100):
    approx_img = Ua @ Sigmaa[:, :,0:k] @ Vta[:, 0:k, :]
```

```
new_img2=np.transpose(approx_img, (1, 2, 0))
new_img2=new_img2-new_img2.min()
new_img2=new_img2/new_img2.max()
```

```
plt.figure()
plt.imshow(new_img2)
plt.imsave('k'+str(k)+'.pdf', new_img2)
```

The results

K=10



K=20



K=50

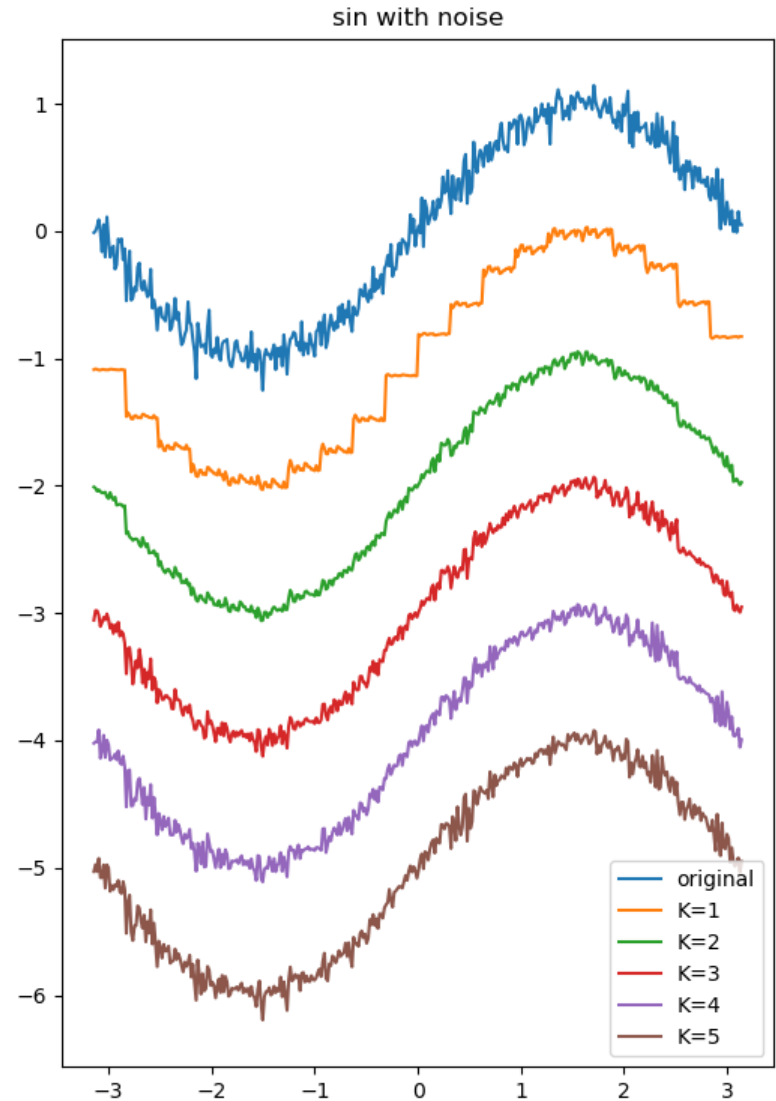
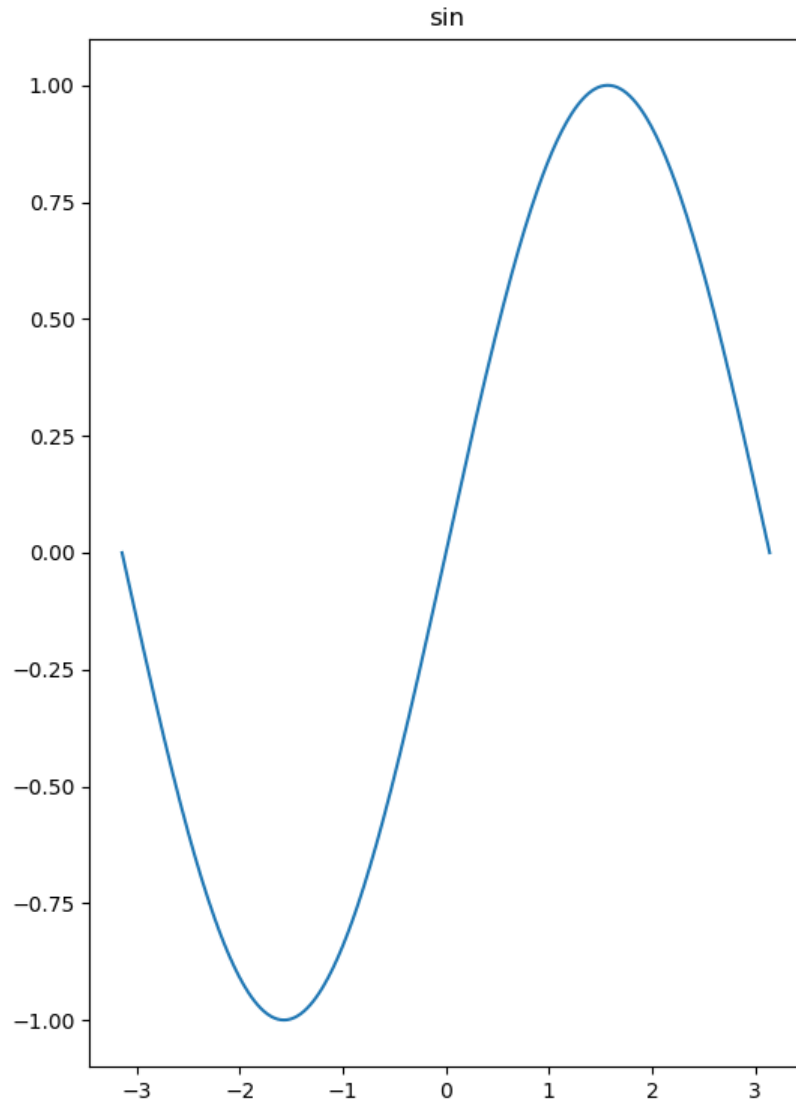


K=100



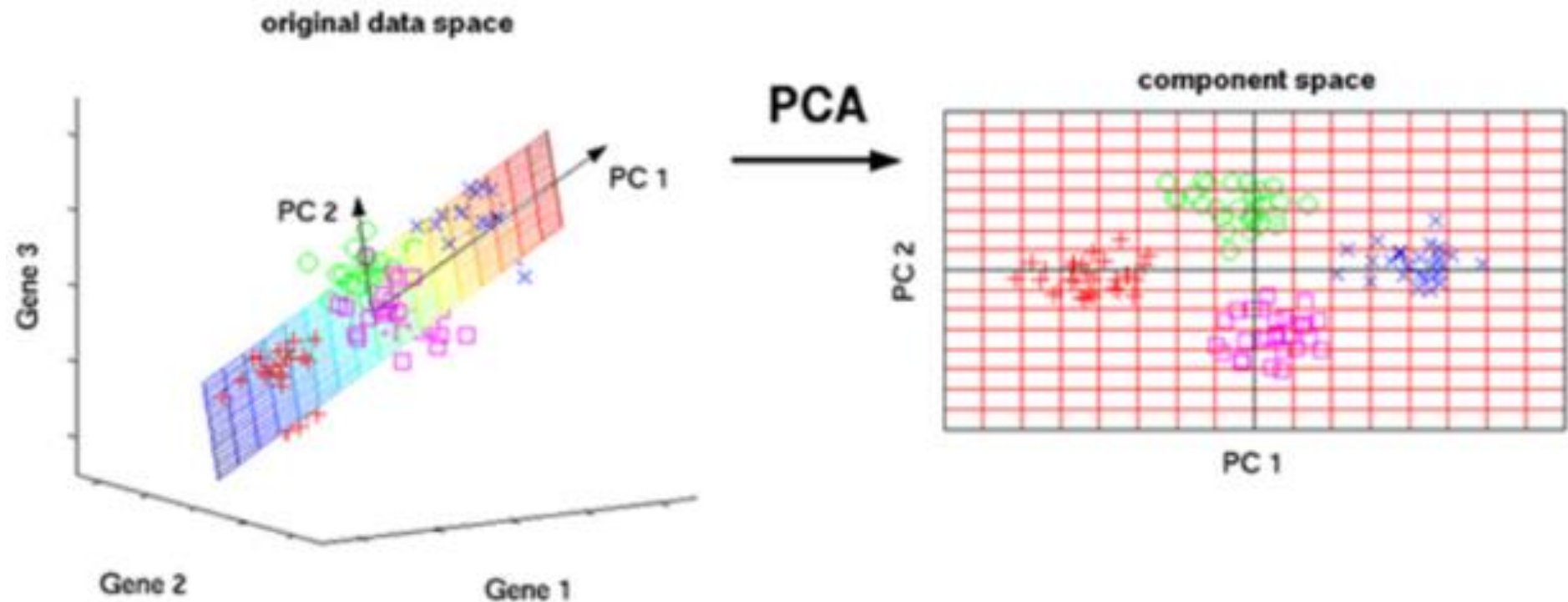
Noise reduction

- In some cases, the noise in the signal corresponds to the states with smaller singular values. Then we can use SVD to reduce the noise.



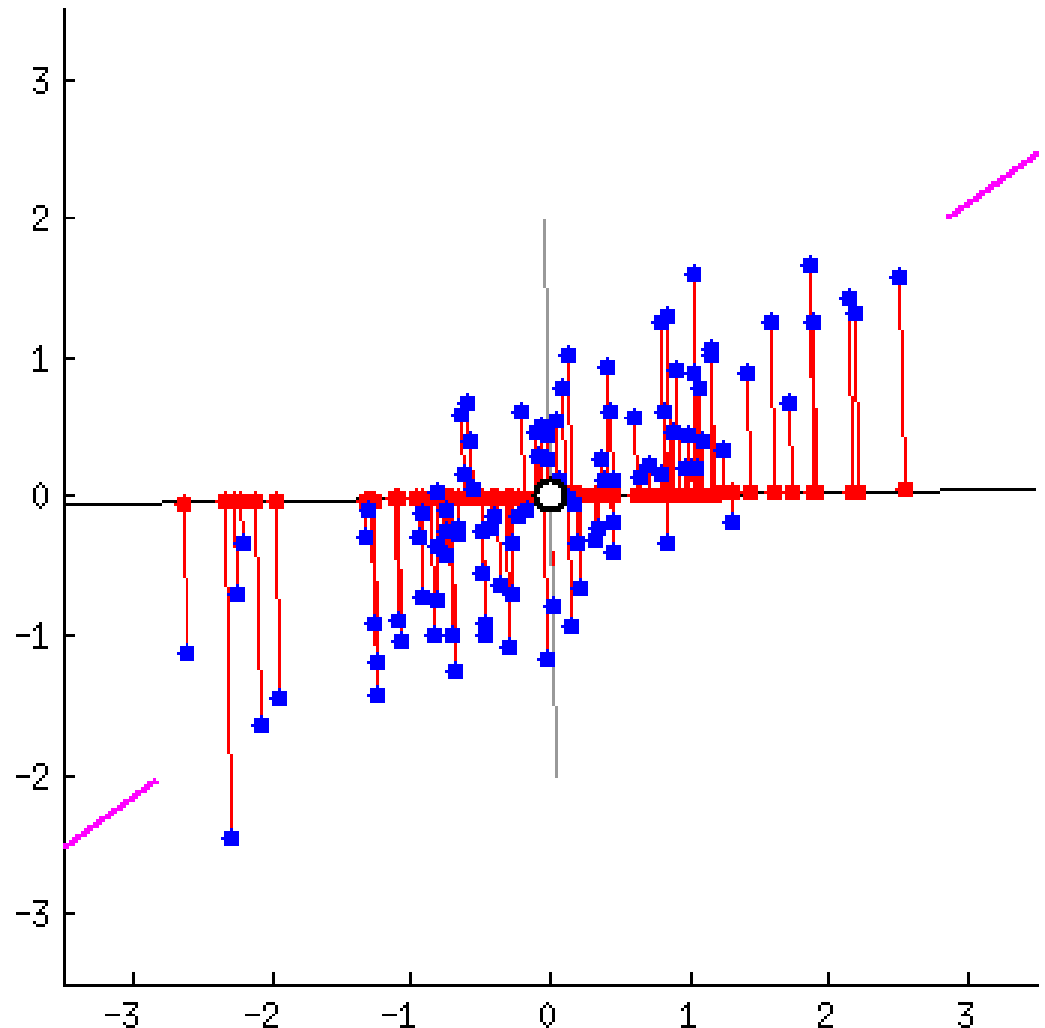
Principal Component Analysis (PCA)

- Principal component analysis, or PCA, is a statistical procedure that allows you to summarize the information content in large data tables by means of a smaller set of “summary indices” that can be more easily visualized and analyzed.



Principal Component Analysis (PCA)

- Statistically, PCA finds lines, planes and hyper-planes in the K-dimensional space that approximate the data as well as possible in the least squares sense.
- A line or plane that is the least squares approximation of a set of data points makes the variance of the coordinates on the line or plane as large as possible.



Variance and covariance

- Variance of one sample X

$$S^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

- Covariance of two samples X and Y

$$\begin{aligned} \text{Cov}(X, Y) &= E[(X - E(X))(Y - E(Y))] \\ &= \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) \end{aligned}$$

- Covariance matrix of multiple samples labelled by X

$$X = \begin{pmatrix} a_1 & a_2 & \cdots & a_m \\ b_1 & b_2 & \cdots & b_m \end{pmatrix}$$

$$\frac{1}{m} X X^T = \begin{pmatrix} \frac{1}{m} \sum_{i=1}^m a_i^2 & \frac{1}{m} \sum_{i=1}^m a_i b_i \\ \frac{1}{m} \sum_{i=1}^m a_i b_i & \frac{1}{m} \sum_{i=1}^m b_i^2 \end{pmatrix} = \begin{pmatrix} \text{Cov}(a, a) & \text{Cov}(a, b) \\ \text{Cov}(b, a) & \text{Cov}(b, b) \end{pmatrix}$$

PCA and EVD

- In PCA, we want to find new basis/principal component, in which all the covariance are minimized.

$$\frac{1}{m}XX^T = \begin{pmatrix} \frac{1}{m} \sum_{i=1}^m a_i^2 & \frac{1}{m} \sum_{i=1}^m a_i b_i \\ \frac{1}{m} \sum_{i=1}^m a_i b_i & \frac{1}{m} \sum_{i=1}^m b_i^2 \end{pmatrix} = \begin{pmatrix} Cov(a, a) & Cov(a, b) \\ Cov(b, a) & Cov(b, b) \end{pmatrix}$$

- Relation of covariance in different basis
- Definition of EVD

$$\begin{aligned} D &= \frac{1}{m}YY^T \\ &= \frac{1}{m}(PX)(PX)^T \\ &= \frac{1}{m}PXX^T P^T \\ &= P\left(\frac{1}{m}XX^T\right)P^T \\ &= PCP^T \end{aligned}$$

$$E^T C E = \Lambda = \begin{pmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_n \end{pmatrix}$$

- **PCA is to just find the EVD of the covariance matrix of all the samples.**

Application of PCA

- **Data Visualization:** When working on any data related problem, Considering that there are a large number of variables or dimensions along which the data is distributed, visualization can be a challenge and almost impossible. Hence, PCA can do that for you since it projects the data into a lower dimension, thereby allowing you to visualize the data in a 2D or 3D space with a naked eye.
- **Machine Learning :** Since PCA's main idea is dimensionality reduction, you can leverage that to speed up your machine learning algorithm's training and testing time considering your data has a lot of features, and the ML algorithm's learning is too slow. PCA can also be used as a machine learning algorithm and it is widely used in quantitative finance, image compression, facial recognition, etc.

Fourier series

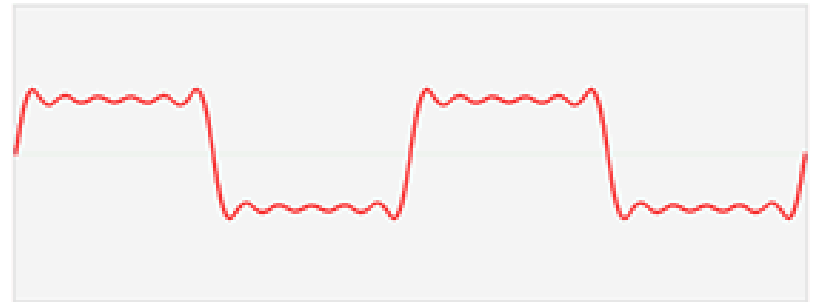
- Consider a periodic real-valued function, $f(x)$, that is integrable on an interval of length P (one period), then $f(x)$ can always be expressed as Fourier series as following

$$f(x) = \frac{a_0}{2} + \sum_{n=1}^N \left(a_n \cos \left(\frac{2\pi}{P} nx \right) + b_n \sin \left(\frac{2\pi}{P} nx \right) \right)$$

$$a_0 = \frac{2}{P} \int_P f(x) dx$$

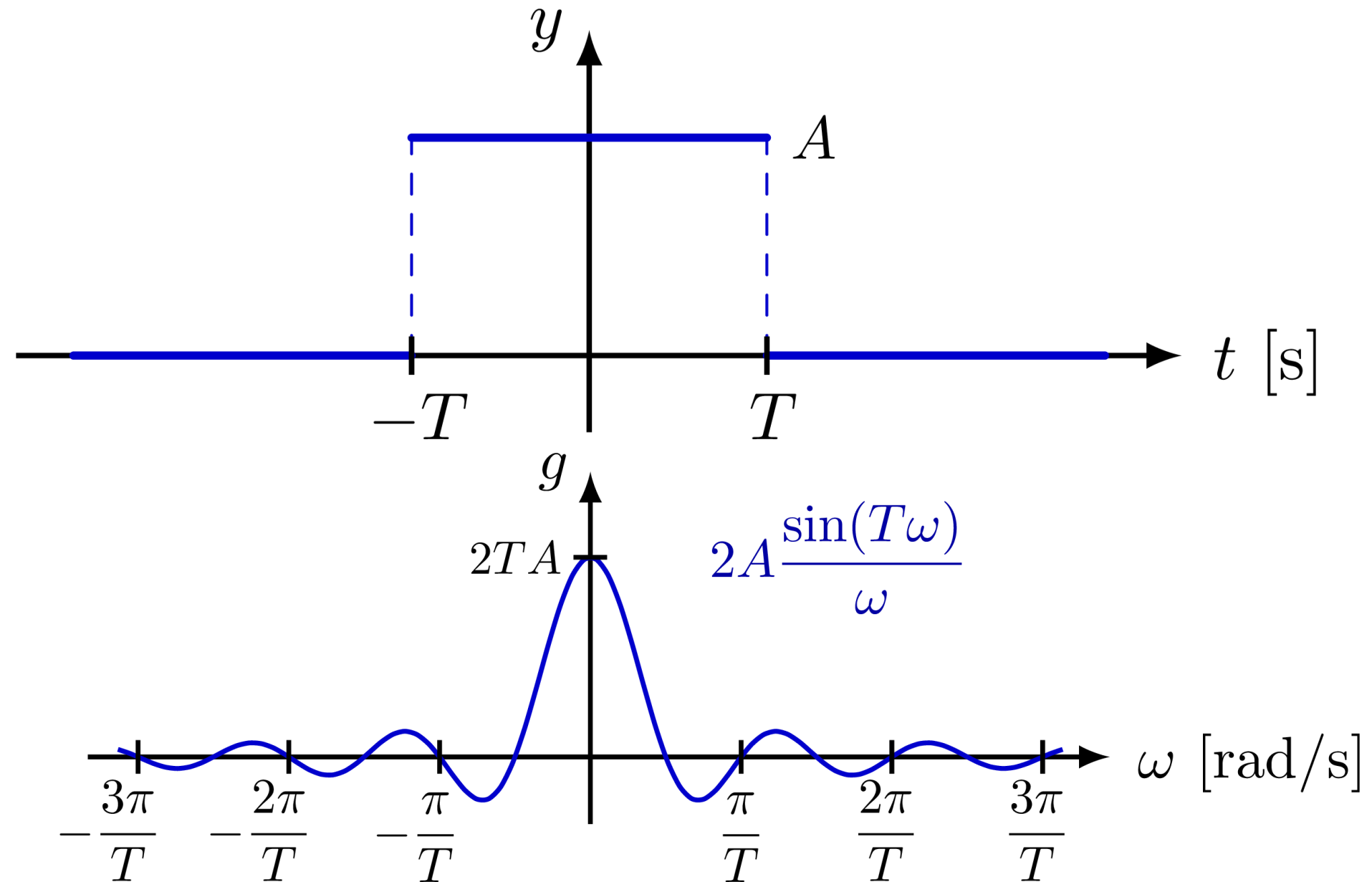
$$a_n = \frac{2}{P} \int_P f(x) \cos \left(\frac{2\pi}{P} nx \right) dx$$

$$b_n = \frac{2}{P} \int_P f(x) \sin \left(\frac{2\pi}{P} nx \right) dx$$



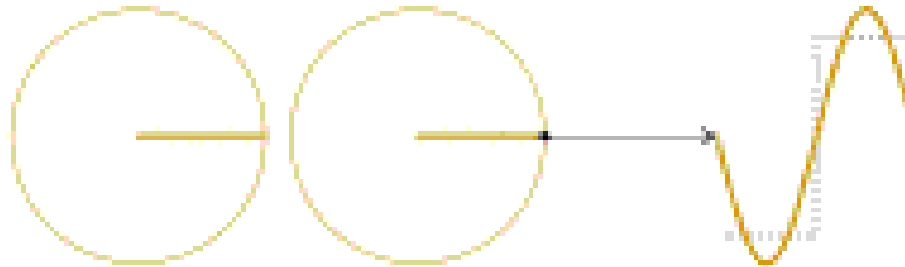
- Usually, we choose $P = [-\pi, \pi]$ or $P = [0, 1]$ to simplify the analysis.

A simple example

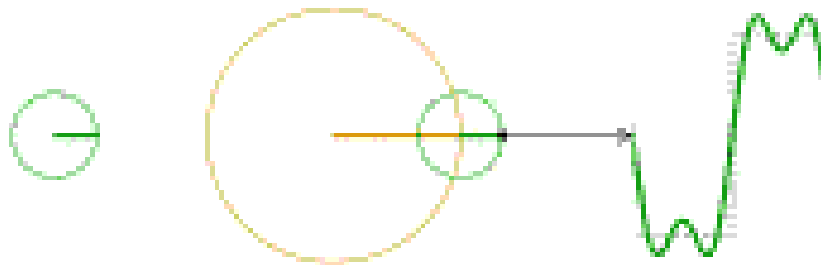


Summation of $\sin(nx)$ $\rightarrow$ circular movement

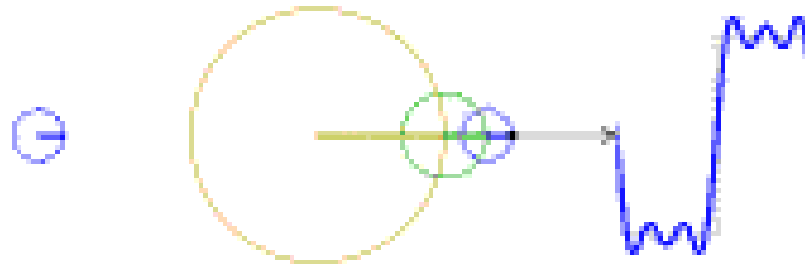
$$\frac{4 \sin \theta}{\pi}$$



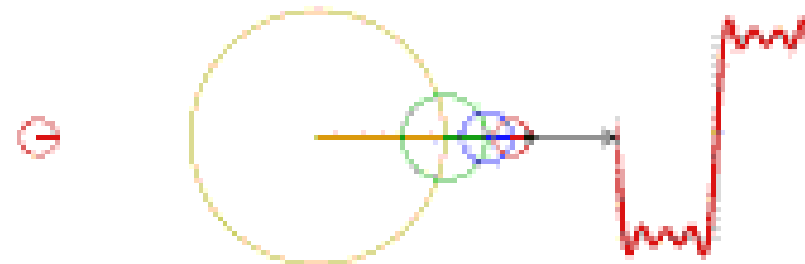
$$\frac{4 \sin 3\theta}{3\pi}$$



$$\frac{4 \sin 5\theta}{5\pi}$$



$$\frac{4 \sin 7\theta}{7\pi}$$



Exponential form (Complex number expression)

- Fourier series can be expressed in simple exponential form

$$f(x) = \sum_{n=-N}^N c_n e^{\frac{i2\pi}{P}nx}$$
$$c_n = \frac{1}{P} \int_P f(x) e^{-\frac{i2\pi}{P}nx} dx$$

- If $f(x)$ is real-valued, then c_n and c_{-n} are complex conjugates, otherwise they are not.
- We can also define the Fourier series for functions of two variables in the square with length P_x and P_y

$$f(x, y) = \sum_{n=-N}^N c_{nm} e^{\frac{i2\pi}{P_x}nx} e^{\frac{i2\pi}{P_y}ny}$$
$$c_{nm} = \frac{1}{P_x P_y} \int_{P_x, P_y} f(x, y) e^{-\frac{i2\pi}{P_x}nx} e^{-\frac{i2\pi}{P_y}ny} dx dy$$

Hilbert space interpretation

- In the language of Hilbert space, the set of function $\{e_n = e^{inx}, n \in \mathbb{Z}\}$ is an orthogonal basis for the space $L^2([-\pi, \pi])$ of square-integrable functions on $[-\pi, \pi]$. This space is actually a Hilbert space with an inner product given for any two elements f and g by $\langle f, g \rangle \equiv \frac{1}{2\pi} \int_{-\pi}^{\pi} f(x) g^*(x) dx$, where $g^*(x)$ is the complex conjugate of $g(x)$. Then the basic Fourier series for Hilbert space can be written as $f(x) = \sum_{n=-\infty}^{\infty} \langle f, e_n \rangle e_n$
- One can demonstrate
$$\frac{1}{\pi} \int_{-\pi}^{\pi} \cos(mx) \sin(nx) dx = 0,$$
$$\frac{1}{\pi} \int_{-\pi}^{\pi} \cos(mx) \cos(nx) dx = \delta_{mn},$$
$$\frac{1}{\pi} \int_{-\pi}^{\pi} \sin(mx) \sin(nx) dx = \delta_{mn}$$

Fourier transform

- The Fourier transform is an extension of the Fourier series that results when the period of the represented function is lengthened and allowed to approach infinity

$$f(x) = \int_{-\infty}^{\infty} c(k) e^{2\pi i k x} dk$$
$$c(k) = \int_{-\infty}^{\infty} f(x) e^{-2\pi i k x} dx$$

- In some cases, we write $\omega = 2\pi k$, then we have

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} c(\omega) e^{i\omega x} d\omega$$
$$c(\omega) = \int_{-\infty}^{\infty} f(x) e^{-i\omega x} dx$$

Discrete Fourier Transform

- The DFT is the equivalent of the continuous Fourier Transform for signals known only at discrete points (usually the case in real life).
- Let $f(x)$ can only exist at given points $x = x_0, x_2, \dots, x_{N-1}$, and we label $f(x_n)$ to be $f(n)$ then

$$f(n) = \frac{1}{N} \sum_{k=0}^{N-1} c(k) e^{i\frac{2\pi}{N}kn} = \frac{1}{N} \sum_{k=0}^{N-1} c(k) w_N^{-kn}$$
$$c(k) = \sum_{n=0}^{N-1} f(n) e^{-i\frac{2\pi}{N}kn} = \sum_{n=0}^{N-1} f(n) w_N^{kn}$$

Where $w_N = e^{-\frac{i2\pi}{N}}$, $n = 0, 1, \dots, N-1$ and $k = 0, 1, \dots, N-1$

Matrix form and FFT

- DFT can be expressed in simple matrix form

$$\begin{bmatrix} c(0) \\ c(1) \\ c(2) \\ c(3) \\ \vdots \\ c(N-1) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & \dots & 1 \\ 1 & w_N^1 & w_N^2 & w_N^3 & \dots & w_N^{(N-1)} \\ 1 & w_N^2 & w_N^4 & w_N^6 & \dots & w_N^{2(N-1)} \\ 1 & w_N^3 & w_N^6 & w_N^9 & \dots & w_N^{3(N-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & w_N^{N-1} & w_N^{2(N-1)} & w_N^{3(N-1)} & \dots & w_N^{(N-1)(N-1)} \end{bmatrix} \begin{bmatrix} f(0) \\ f(1) \\ f(2) \\ f(3) \\ \vdots \\ f(N-1) \end{bmatrix}$$

- It is discovered that the Fourier matrix W_N has the following properties due to its high symmetry

$$W_{2N} = \begin{bmatrix} I_N & D_N \\ I_N & -D_N \end{bmatrix} \begin{bmatrix} W_N & 0 \\ 0 & W_N \end{bmatrix} P_{2N}$$

Where I_N is an identity matrix, D_N is a diagonal matrix with diagonal element to be $w_{2N}^0, w_{2N}^1, \dots, w_{2N}^{N-1}$ and P_{2N} is a permutation matrix.

- By continuously doing this matrix factorization, you can get the Fast Fourier transform with complexity as $N \log N$ instead of N^2 .

Typical Application of FFT

The FFT is used in digital recording, sampling, additive synthesis and pitch correction software. The FFT's importance derives from the fact that it has made working in the frequency domain equally computationally feasible as working in the temporal or spatial domain. Some of the important applications of the FFT include:

- Filter, denoise, and compress data
- Solving difference equations
- Fast large-integer and polynomial multiplication
- Efficient matrix–vector multiplication for structured matrices
- Fast algorithms for discrete cosine or sine transforms (e.g. fast DCT used for JPEG and MPEG/MP3 encoding and decoding)
- Modulation and demodulation of complex data symbols for 5G, LTE, Wi-Fi, DSL, and other modern communication systems

Plot Fourier using FT



Monte Carlo method or random process

Monte Carlo methods, or Monte Carlo experiments, are a broad class of computational algorithms that rely on repeated **random sampling** to obtain numerical results. The underlying concept is to **use randomness to solve problems that might be deterministic in principle.**

MC methods are often used in physical and mathematical problems and are most useful when it is difficult or impossible to use other approaches. Monte Carlo methods are mainly used in three problem classes: optimization, numerical integration, and generating draws from a probability distribution.

In this lecture, we will show some example to use MC methods or random process in real life problems.

One Example: AlphaGO

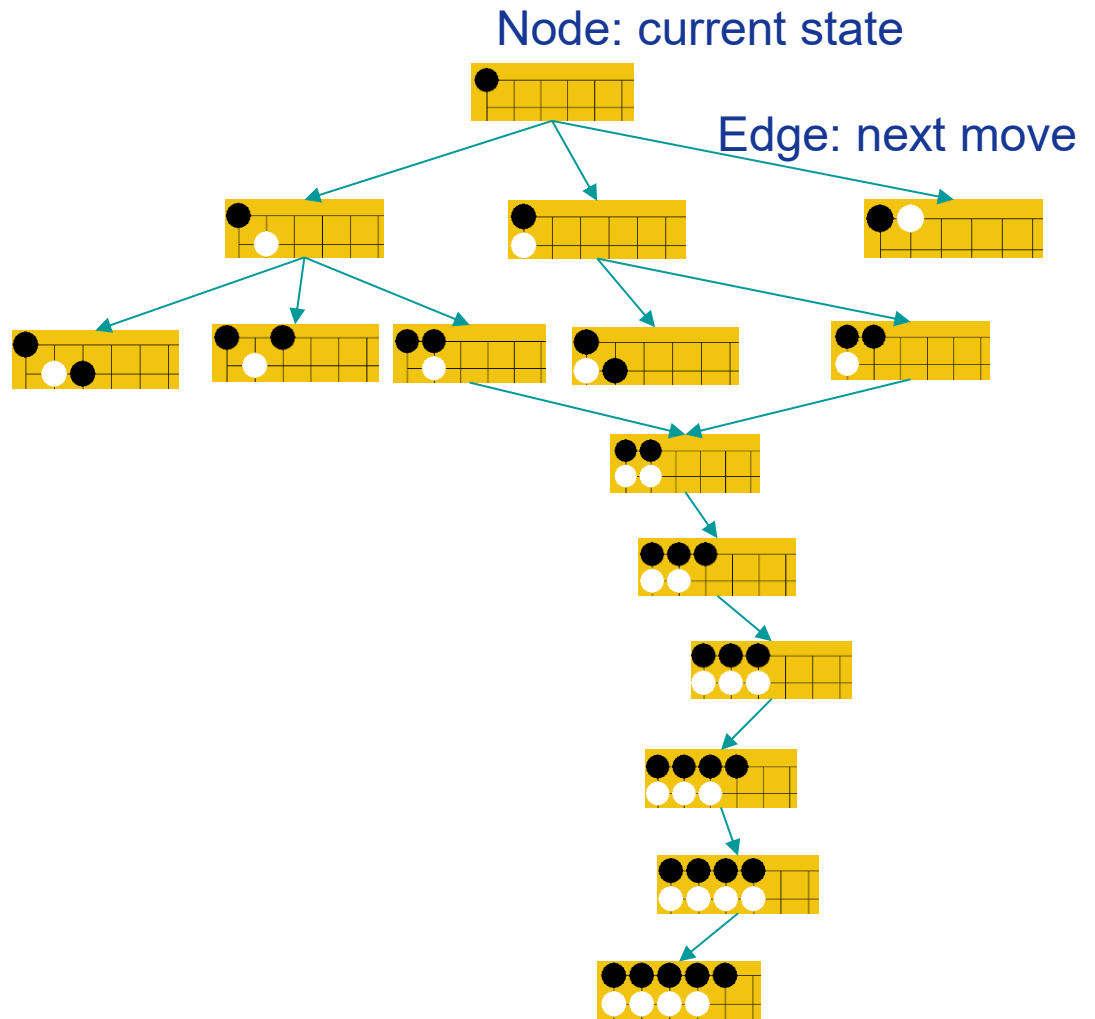


Deep learning →
Action-value
function

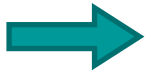
Monte Carlo Tree
Search →
Optimized move

Game tree

In game theory, a game tree is a directed graph whose **nodes** are positions in a game and whose **edges** are moves. The complete game tree for a game is the game tree starting at the initial position and containing all possible moves from each position

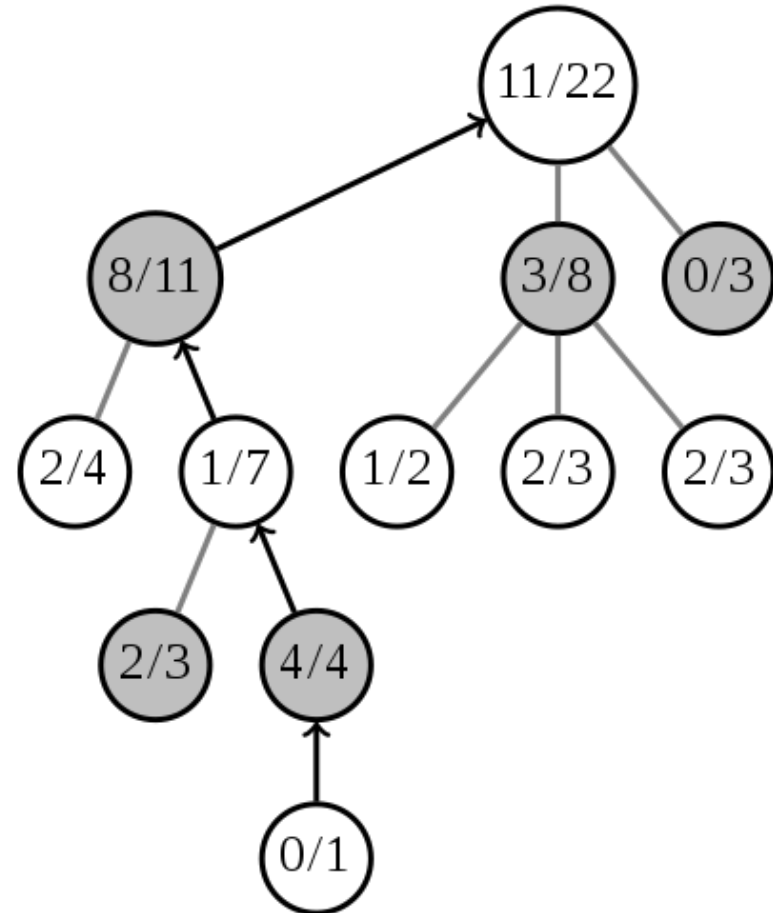


Game tree with winning probability



n: number of win
m: number of play

Every status could be taken as a node which can be assigned with some winning probability. And different statuses (nodes) are connected by different moves.



Monte Carlo tree search (MCTS)

MCTS is a **heuristic(启发式的)** search algorithm for some kinds of **decision processes**, most notably those employed in game play. MCTS was introduced in 2006 for computer Go. It has been used in other board games like chess and shogi, games with incomplete information such as bridge and poker, as well as in real-time video games (such as Total War: Rome II's implementation in the high level campaign AI).

The focus of MCTS is on the analysis of the most promising moves, expanding the search tree based on **random sampling of the search space**. The application of Monte Carlo tree search in games is based on many playouts also called roll-outs. In each playout, the game is played out to the very end by selecting moves at random. The final game result of each playout is then used to weight the nodes in the game tree so that better nodes are more likely to be chosen in future playouts.

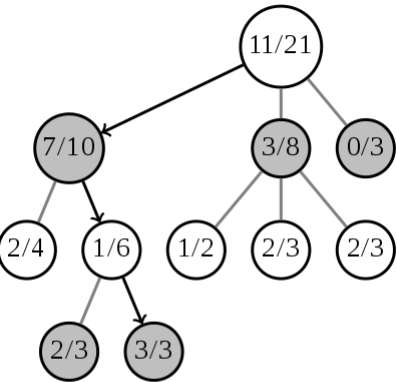
Pure Monte Carlo Game Search (PMCGS)

PMCGS is the most basic way to use playouts, where one applies the same number of playouts after each legal move, then chooses the move leading to the most victories. The more playouts, the more accurate. Each round of PMCGS consists of four steps:

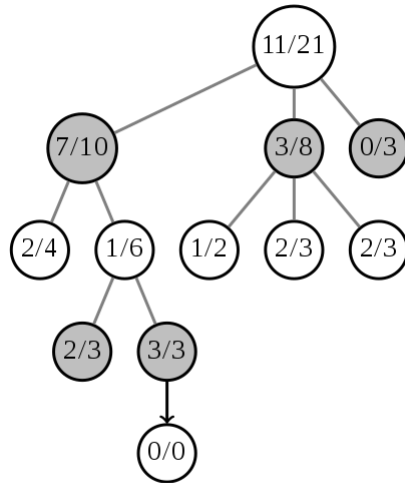
1. Selection: start from root R and select successive child nodes until a leaf node L is reached. The root is the current game state and a leaf is any node from which no simulation (playout) has yet been initiated.
2. Expansion: unless L ends the game decisively (e.g. win / loss / draw) for either player, create one (or more) child nodes and choose node C from one of them.
3. Simulation: complete a playout from node C. A playout may be as simple as choosing uniform random moves until the game ends
4. Backpropagation: use the result of the playout to update information in the nodes on the path from C to R.

Steps of Monte Carlo tree search

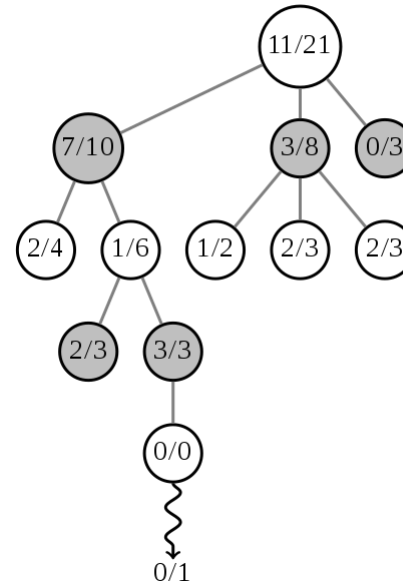
Selection



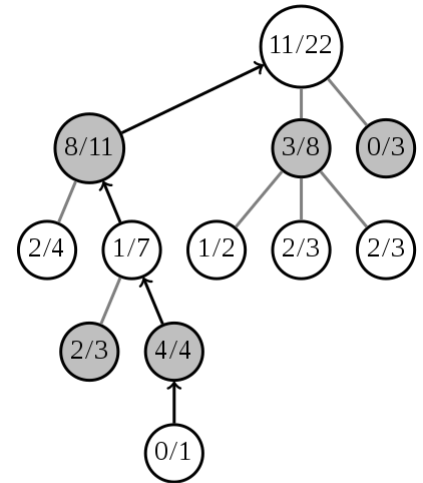
Expansion



Simulation



Backpropagation



Select one node using random number based on the probability of winning.

Expand the node by randomly choosing a legal move.

Randomly choose the following moves until the game ends

Use the result of the playout to update information for all the previous nodes

More discussion

- The main difficulty in selecting child nodes is maintaining some balance between the exploitation of deep variants after moves with high average win rate and the exploration of moves with few simulations.
- Pure Monte Carlo tree search does not need an explicit evaluation function. Simply implementing the game's mechanics is sufficient to explore the search space (i.e. the generating of allowed moves in a given position and the game-end conditions). As such, Monte Carlo tree search can be employed in games without a developed theory or in general game playing.
- A disadvantage is that, in a critical position against an expert player, there may be a single branch which leads to a loss. Because this is not easily found at random, the search may not "see" it and will not take it into account

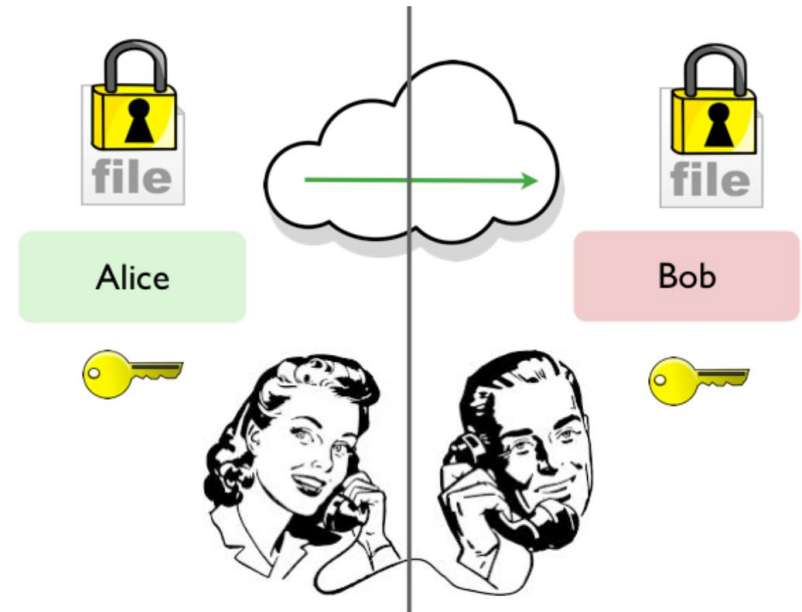
The other example: encryption

- In many cases, we want our information or message to be safe, then we need encryption method.

- **Symmetric-key cryptography**



- **Asymmetric-key cryptography**



Simplest symmetric-key algorithm

- A simple example is the ***substitution cipher***: take the letters of the alphabet and add a constant to them, e.g. if the constant is three, then you change A to D, B to E, C to F and so forth. Decoding the message just changes D to A, E to B, F to C and so forth.

Example codes

```
from numpy import mod
shift=1
all_characters='abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'12
34567890-=[]\,./~!@#$%^&*()_+{}|:<>? '
num_char=len(all_characters)
codes={}; decodes={}
for ni in range(num_char):
    codes[all_characters[ni]]=all_characters[(ni+shift)%num_char]
    decodes[all_characters[mod(ni+shift,num_char)]] =all_characters[ni]

str1='we are learning encryption now. Do you understand it? Easy!!!'
str2=[];str3=[]
for ni in range(len(str1)):
    str2.append(codes[str1[ni]])

for ni in range(len(str2)):
    str3.append(decodes[str2[ni]])

print("The original message is  --->\n","".join(str1))
print("After coding, message is  --->\n","".join(str2))
print("After decoding, message is --->\n", "".join(str3))
```

It is very easy to break. You only need to try less than 100 times. In modern cryptography, the rule is to assume the worst: even the thief knows the principles behind the cipher you are using, they will not be able to read your message.

Better substitution cipher

- Instead of adding the same constant to each letter no matter where it is in your message, let us use different constants for letters in different positions of your message.

```
from numpy import mod
import numpy as np
```

```
def char_table(str,shift):
```

```
all_characters='abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ`1234567890-=[\],./~!@#$%^&*()_+{}|:<>? '
```

Example codes-part2

```
def coding_decoding(str,codes,coding=1):
```

```
    str2=[]
```

```
    for ni in range(len(str)):
```

```
        str2.append(char_table(str[ni],coding*codes[ni]))
```

```
    return str2
```

```
np.random.seed(10)
```

```
str1='much better methods'
```

```
codes=[];
```

```
for ni in range(len(str1)):
```

```
    codes.append(ni)
```

```
str2=coding_decoding(str1,codes,1)
```

```
str3=coding_decoding(str2,codes,-1)
```

```
print("The original message is  --->\n","".join(str1))
```

```
print("After coding, message is  --->\n","".join(str2))
```

```
print("After decoding, message is --->\n", "".join(str3))
```

It is still not very safe. You have to send “codes” to your friends. Once others have the codes, they can decode your message.

Further improvement: Random cipher

- The best way to create a varying cipher is to use random number!!! If all the numbers in the constant sequence is random, it is very hard to guess what the next number in the sequence will be. They need to try all possible combinations. For example, the message has M characters and the number of characters in the character table is N , then they need to try around N^M times!
- You can make one-time pad using real random numbers, while this method has a big disadvantage: you have to create the pad and get it to your friend without anyone else seeing it, also both parties have to guard it very carefully against theft, and it can only be used once.
- A much better choice is to use pseudorandom numbers. You do not need the entire pad to use it, and you just need to know the seed for the random number generator, which is called a **key**.
- Making a good secret code and making a good random number generator are basically the same problem.

Example codes: part1

```
from numpy import mod
from random import random,seed,randrange

def char_table(str,shift):

all_characters='abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ`1234567890-=[]\;.,/~!@#$%^&*()_+{}|:<>?'
    num_char=len(all_characters)
    pos=all_characters.find(str)
    return all_characters[mod(pos+shift,num_char)]

def coding_decoding(str,codes,coding=1):
    str2=[]
    full_codes=codes
    if len(str)>len(codes):
        N_codes=len(codes)
        for ni in range(N_codes,len(str)):
            full_codes.append(codes[mod(ni,N_codes)])
    for ni in range(len(str)):
        str2.append(char_table(str[ni],coding*full_codes[ni]))
    return str2
```

Example codes: part2

```
str1='I love hkust when i was a 7-year old child!'
```

```
key=1234567
seed(key)
codes=[];
codes_len=len(str1);
codes_range=1000;
for ni in range(codes_len):
    codes.append(randrange(codes_range))
```

```
#codes=[1]
str2=coding_decoding(str1, codes,1)
str3=coding_decoding(str2, codes,-1)
```

```
print("The original message is --->\n","".join(str1))
print("After coding, message is --->\n","".join(str2))
print("After decoding, message is --->\n", "".join(str3))
```

It is still not good enough. In many cases, we need to use asymmetric cryptography

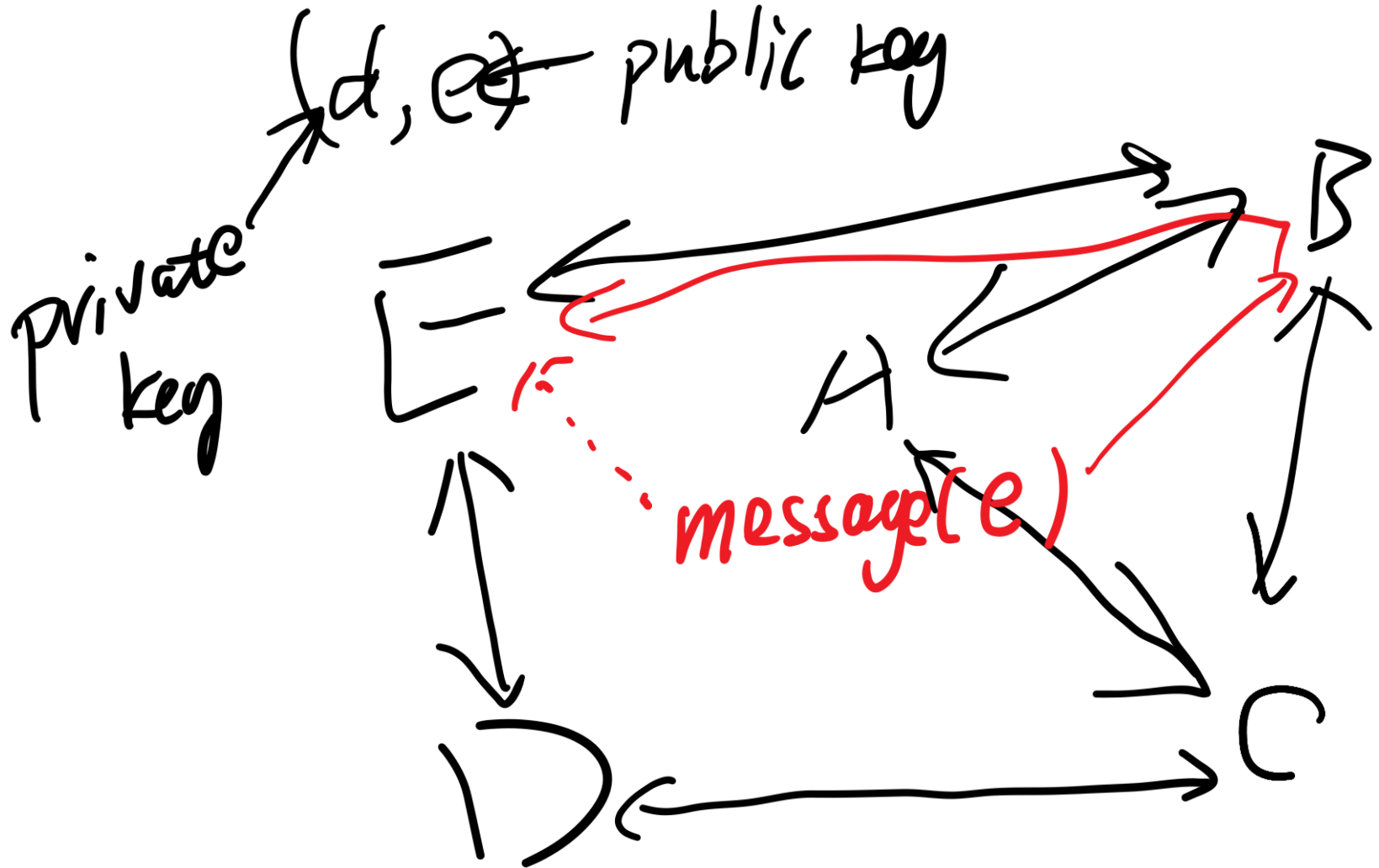
RSA—asymmetric cryptography

Key generation (n,e,d)

1. Choose two distinct prime numbers p, q . **Both are kept secret.**
2. Compute $n = p * q$. n is used as the modulus for both the public and private keys. n is released as part of the **public key**.
3. Compute $\phi(n) = (p - 1) * (q - 1)$. **$\phi(n)$ is kept secret.**
4. Choose an integer e such that $1 < e < \phi(n)$ and e and $\phi(n)$ should be are coprime.
 e having a short bit-length and small hamming weight results in more efficient encryption – the most chosen value is $e = 2^{216} + 1 = 65537$. e is released as part of the **public key**.
5. Determine d as $d * e \bmod \phi(n) = 1$. d can be computed efficiently by using the Extended Euclidean algorithm. d is kept secret as the **private key** exponent.

Encryption and decryption

- The public key is (n, e) . The message m can be encrypted as $c = m^e \bmod n$.
- The private key is (n, d) . The encrypted message c can be decrypted as $m' = c^d \bmod n$.
- You can prove that as long as $m < n$, then $m' = m$
- RSA is very simple and useful. It has been generally used in **encrypted communication** and **digital signature**.
- The disadvantage of RSA is that it is not as fast as symmetric cryptography. So in cases of communicating large amount of messages, we combine both methods. The messages are encrypted using symmetric methods, while the key used in the symmetric cryptography is further encrypted using RSA.



$$A \xrightarrow{25} B \xrightarrow{25} E$$

$$A: 25 \xrightarrow{(n,e)} 16 \rightarrow B(n,e)$$

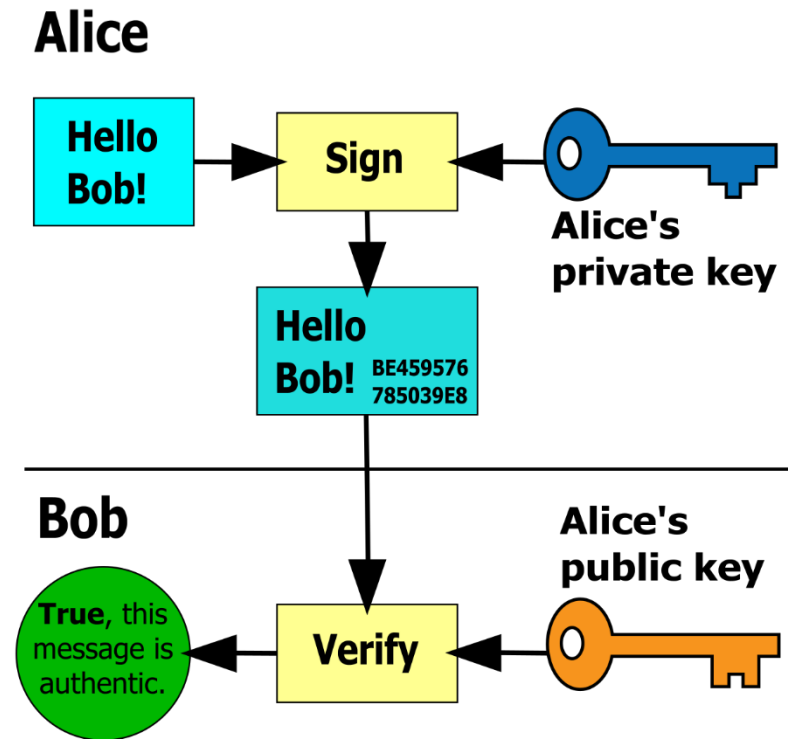
(n,e)

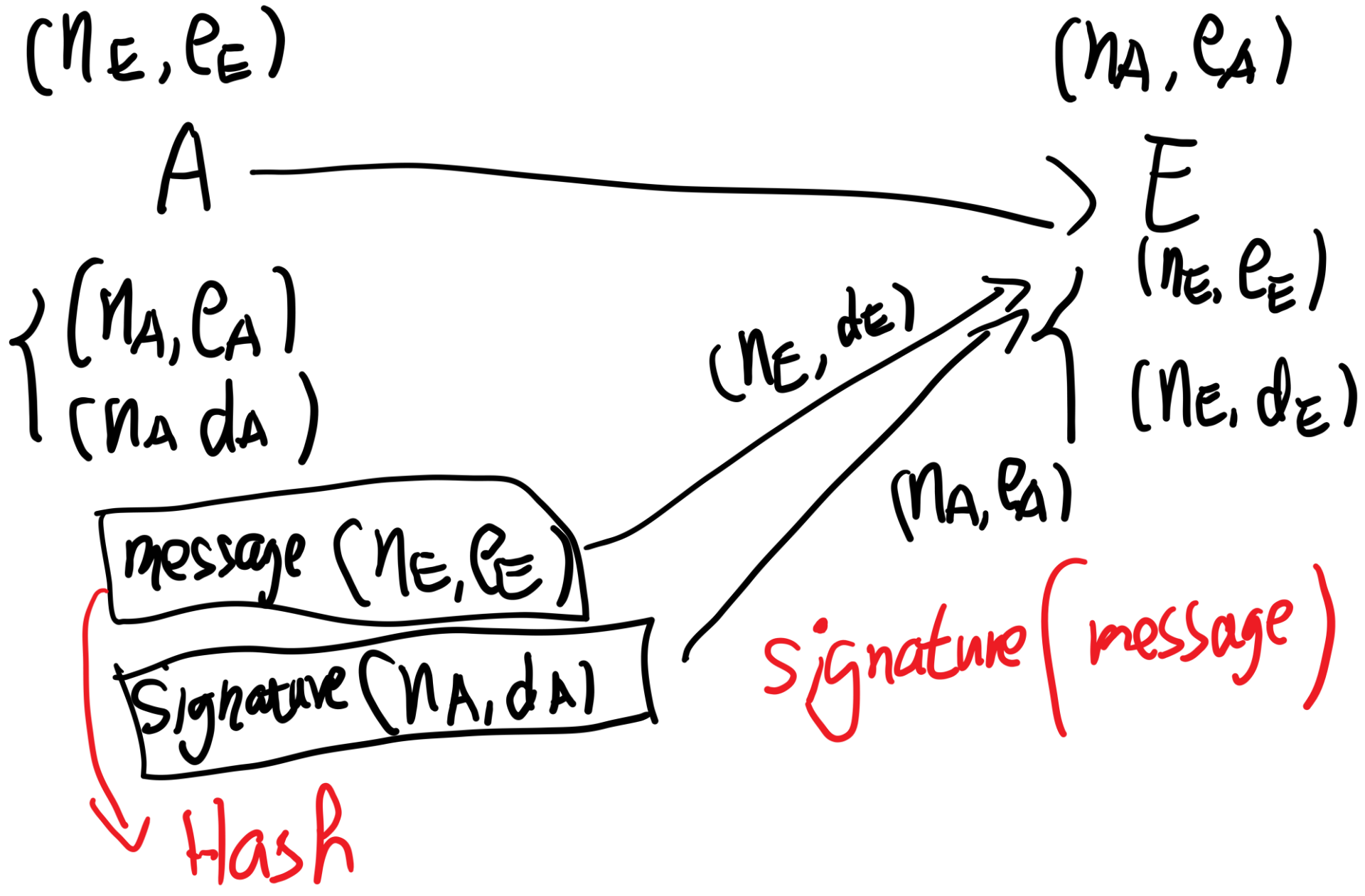
$$25 \leftarrow \underline{(n,d)} 16 \leftarrow E \xrightarrow{\text{generate } (n,e,d)}$$

$\downarrow 16, 17$

Digital signature

- To **encrypt message**, the message is **encrypted** using **public key** by others and then the encrypted message is sent to me. I can further use my **private key** to **decrypt** the encrypted message to get the original message.
- In **digital signature**, the process is opposite. I **encrypted** my signature using my **private key**, and others could use my **public key** to **verify** whether the signature is from me.





Hash Algorithm

A hash function is any function that can map data of arbitrary size to fixed-size values. The values returned by a hash function are called hash values, hash codes, digests, or hashes. The values are used to index a fixed-size table called a hash table.

The ideal cryptographic hash function has five main properties:

- It is deterministic so the same message always results in the same hash
- It is quick to compute the hash value for any given message
- It is infeasible to generate a message from its hash value except by trying all possible messages
- A small change to a message should change the hash value so extensively that the new hash value appears uncorrelated with the old hash value
- It is infeasible to find two different messages with the same hash value

Secure Hash Algorithms (SHAs)

- SHAs are a family of cryptographic hash functions published by the National Institute of Standards and Technology (NIST) as a U.S. Federal Information Processing Standard (FIPS). There are many different classes of functions. We will take SHA-256 or SHA-512 as an example.
- In SHA-256 or SHA-512, all the input string will be converted to be an binary number with 256 or 512 bits. You can easily use SHA-256 or SHA-512 in Python.

Using SHA in python

```
from hashlib import sha256
#from hashlib import sha512

data = input('Enter plaintext data: ')
output = sha256(data.encode('utf-8'))

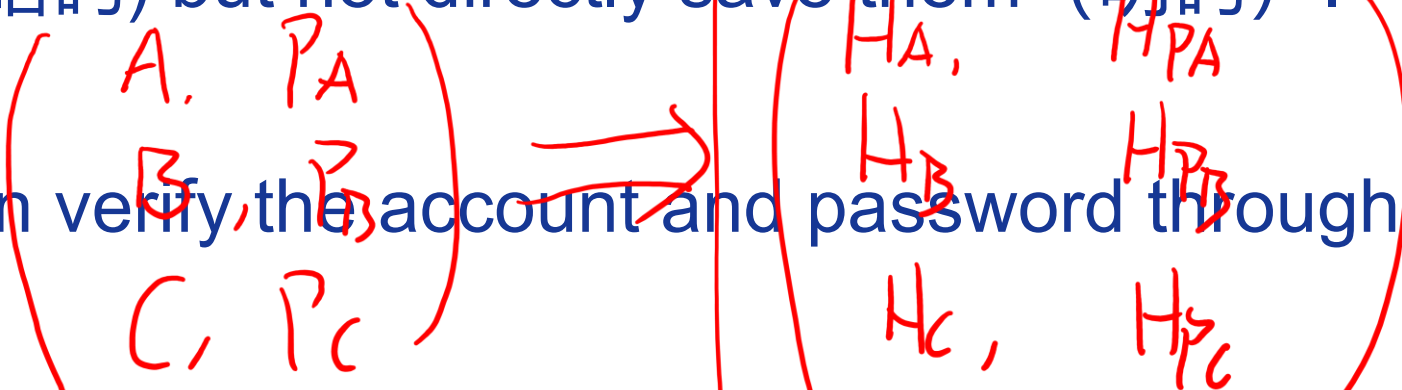
#show the results in different format
val=output.hexdigest();
int_val=int(val,16);
binary_val=bin(int_val)
```

You can also compare your results with the results from the following website <https://www.movable-type.co.uk/scripts/sha256.html>

An simple example: save password safely

The idea is to save both account and password in SHA value (暗码) but not directly save them (明码) .

One can verify the account and password through SHA.



A, P_A	H_A , H_{P_A}
B, P_B	H_B , H_{P_B}
C, P_C	H_C , H_{P_C}

Advantage: one cannot access even he gets the full password table. It is much more safe and all the private information is well protected.

One minor disadvantage: the total number of account is only 2^{256} or 2^{512} . Usually, it should be large enough except you have a really big company (^o^)

The example code

```
from hashlib import sha256

def func_register(acnt,pwd,pwd_table={}):
    sha_acnt = sha256(acnt.encode('utf-8'));    sha_pwd = sha256(pwd.encode('utf-8'))
    if sha_acnt.hexdigest() in pwd_table:
        print(acnt,'has been used, please try another one')
    else:
        pwd_table[sha_acnt.hexdigest()]=sha_pwd.hexdigest()
    return pwd_table

pwd_table={}; pwd_table=func_register('student','Excellent',pwd_table)

acnt=input('Enter your acnt:\n'); pwd=input('Enter your password:\n')
sha_acnt = sha256(acnt.encode('utf-8')); sha_pwd = sha256(pwd.encode('utf-8'))
if sha_acnt.hexdigest() in pwd_table:
    if pwd_table[sha_acnt.hexdigest()]==sha_pwd.hexdigest():
        print('your password is right')
    else:
        print('your password is wrong')
else:
    print(acnt,'does not exist!')
```

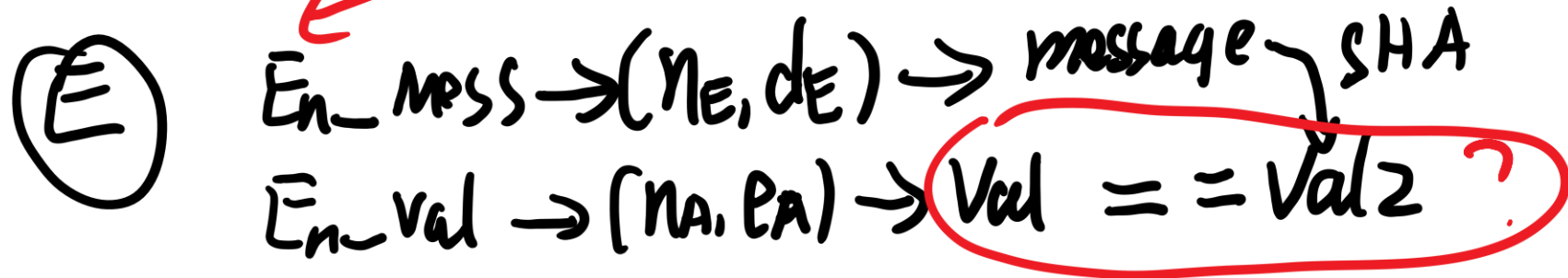
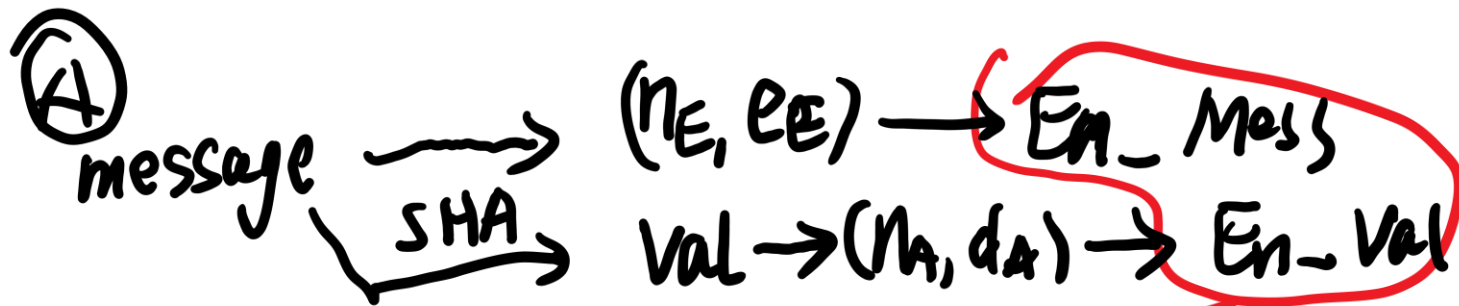
Why is it very safe?

- To get the text (明码), which has the same hash value. You need to try infinite number of times.
- To get the right hash value (暗码) . You need to try $2^{256} = 115792089237316195423570985008687907853269984665640564039457584007913129639936$ or $2^{512} = 13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946433649006084096$ times

Encrypted communication

- To build a really safe communication, we need two pairs of keys from both communicating sides, and use HASH methods.
- For example, A wants to send a message to B. A will use B's public key to encrypt the message and use A's private key to encrypt A's signature, and A send both encrypted message and signature to the internet.
- After B receives both encrypted message and signature from the internet, B could check whether it is from A through A's public key to verify A's signature. If so, B can use B's private key to decrypt the encrypted message to get the original message.

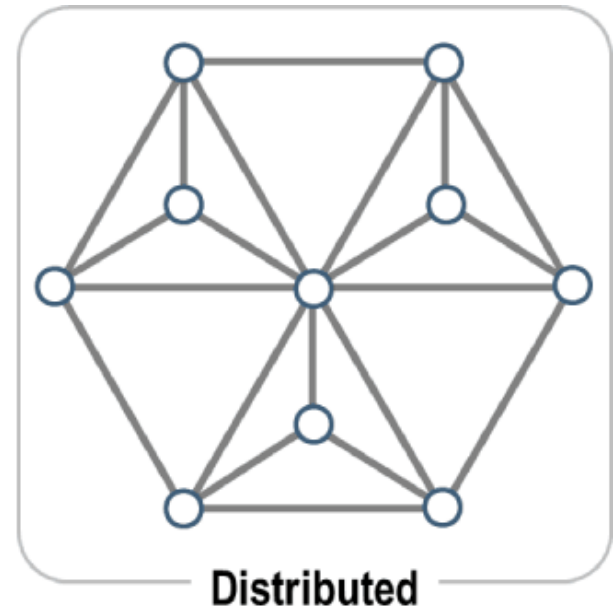
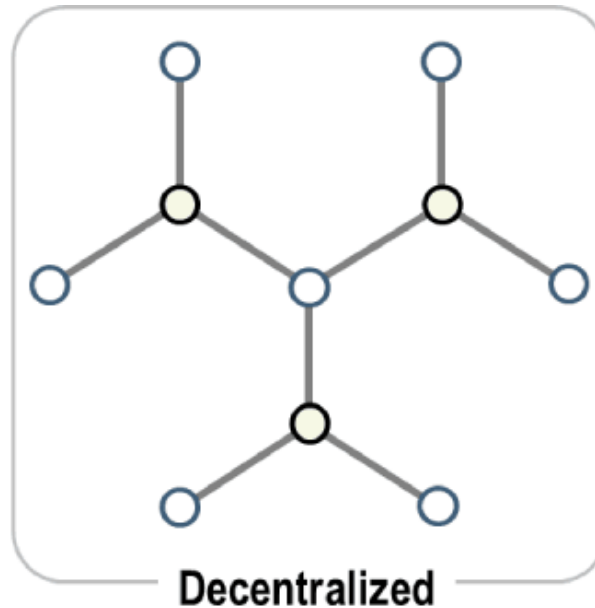
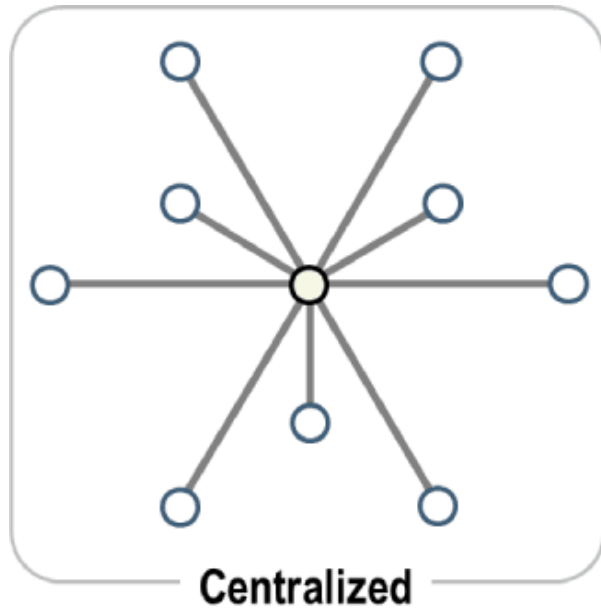
Completed Encrypted communication



Block Chain and Bitcoin

- A block chain is a growing list of **records**, called **blocks**, that are linked using cryptography. Each block contains a cryptographic **hash of the previous block**, a **timestamp**, and **transaction data** (generally represented as a Merkle tree).
- **Bitcoin** (฿) is a cryptocurrency. It is a **decentralized** digital currency without a central bank or single administrator that can be sent from user to user on the **peer-to-peer** bitcoin network without the need for **intermediaries**. Transactions are verified by **network nodes** through cryptography and recorded in a public distributed ledger called a **block chain**.

Structure of network

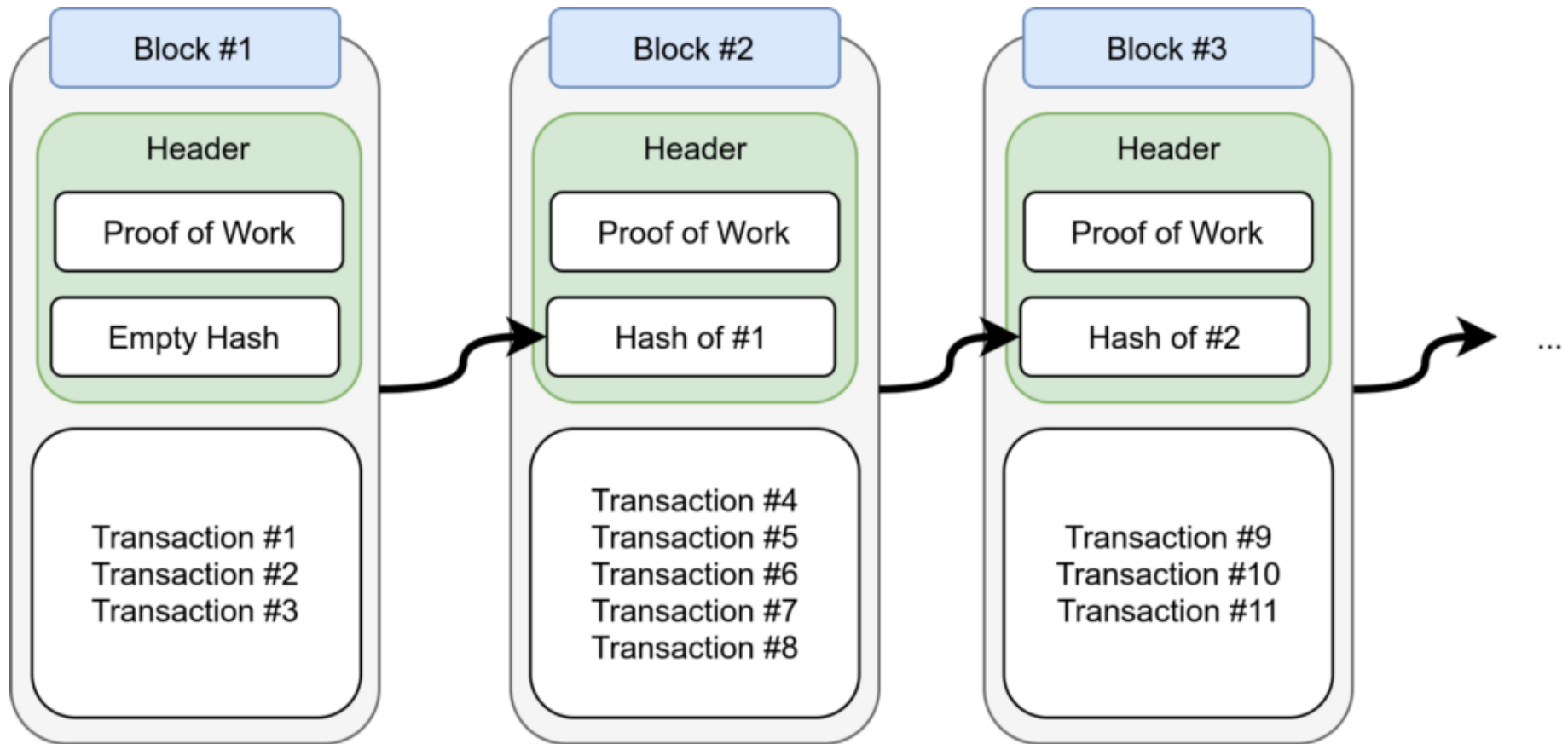


Centralized. One center has high accessibility and thus represents the dominant element of the network and the spatial structure it supports.

Decentralized. Although the center is still the point of highest accessibility, the network is structured so that sub-centers have also significant levels of accessibility.

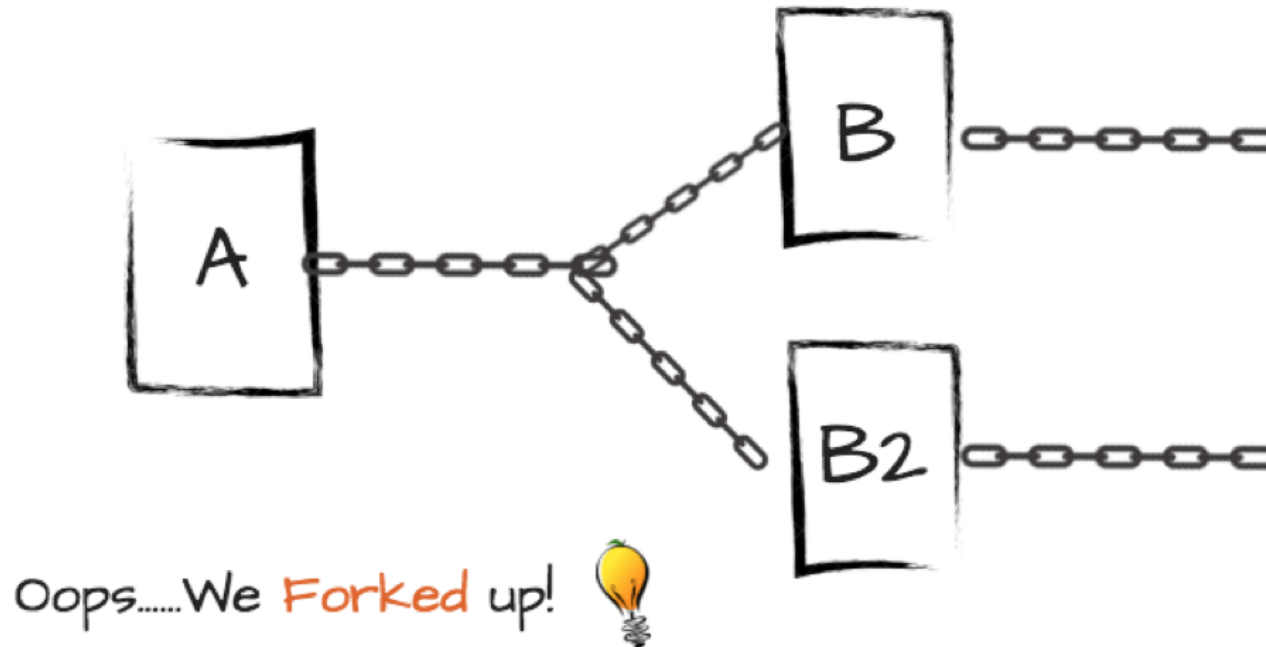
Distributed. No center has a level of accessibility significantly different from the others, which implies a high connectivity levels and redundancy.

Basic structure of blockchain



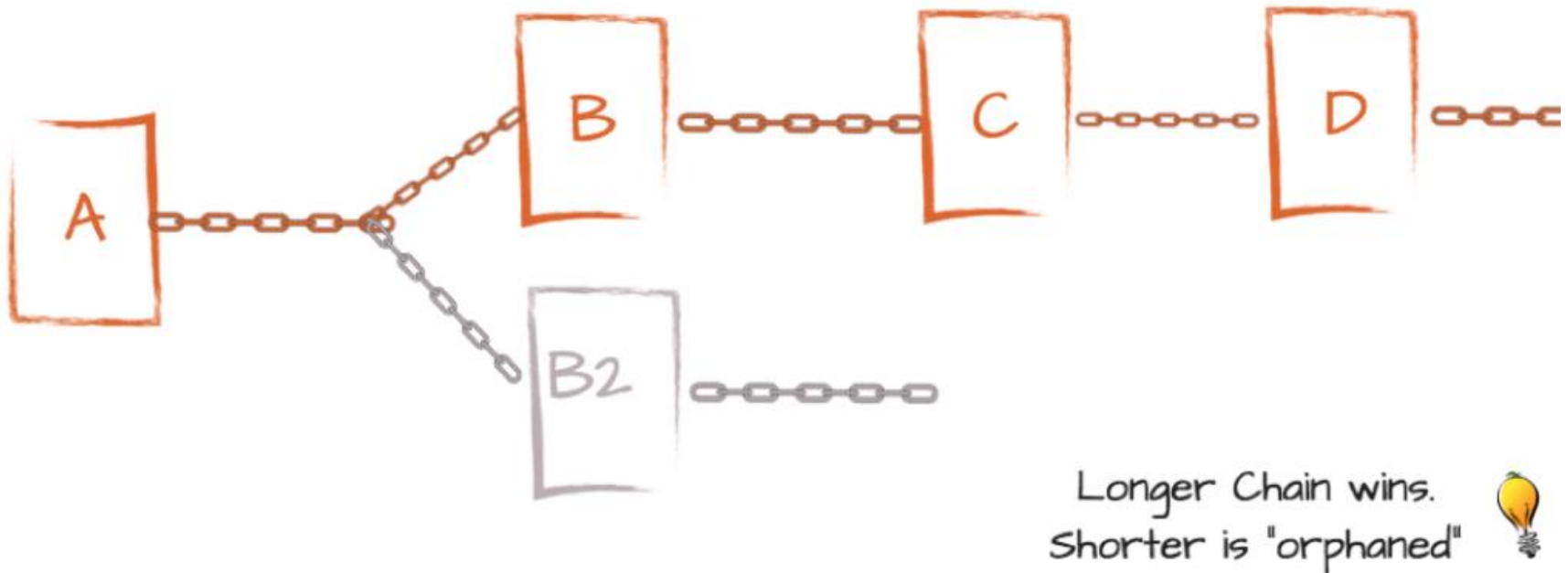
- We can use data to save different records or transactions
- These data cannot be changed
- Perfect ledger in the sense of security

Proof of work



- The prevent adding two blocks at the same time, we **artificially** make it difficult
- Proof of work: by adding a nonce (number only used once) to the block to ensure the Hash values smaller than a certain number (e.g., the first few digits to be zero).

Long chain is the right chain



- The Longest Chain Rule ensures that network will recognise the “chain with most work” as the main chain. The chain with the most work is typically (not always) the longest of the forks.
- All the blocks & transactions on other shorter chains will be considered orphaned and will be ignored.